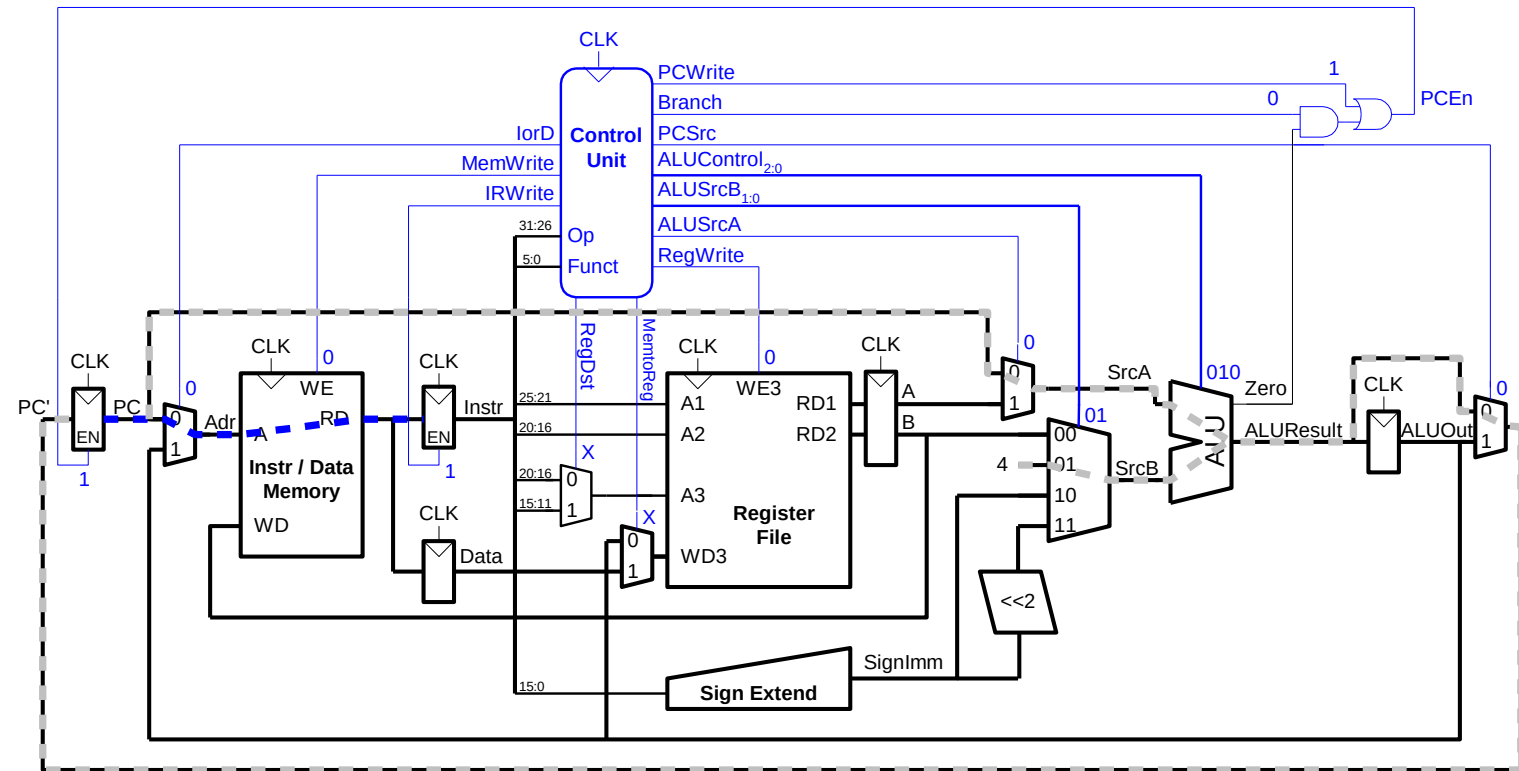
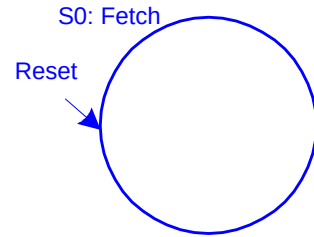
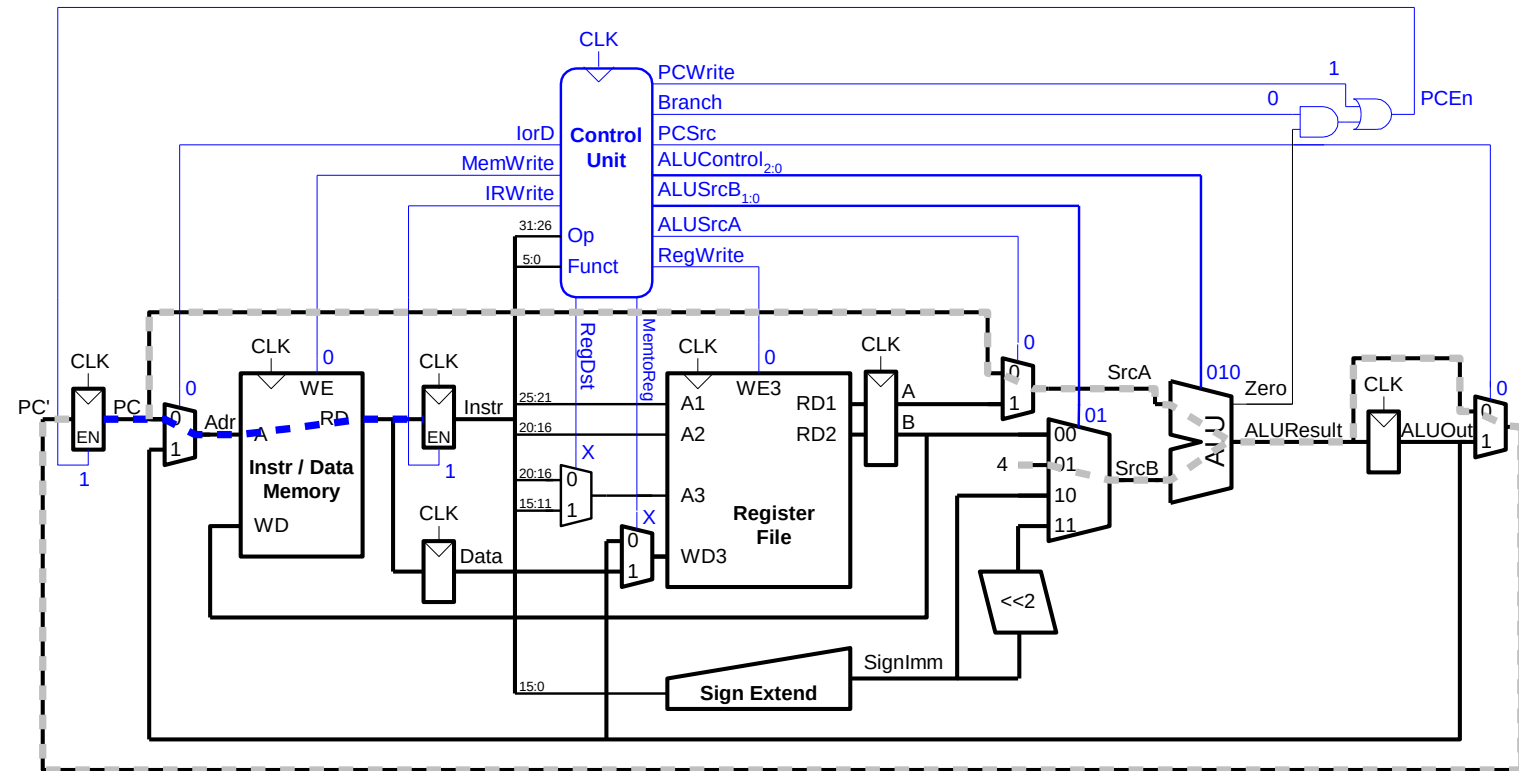
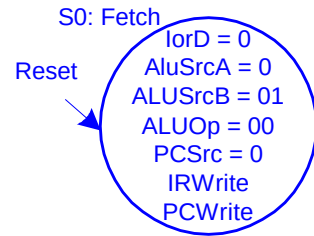


Multi-Cycle Microarchitectures

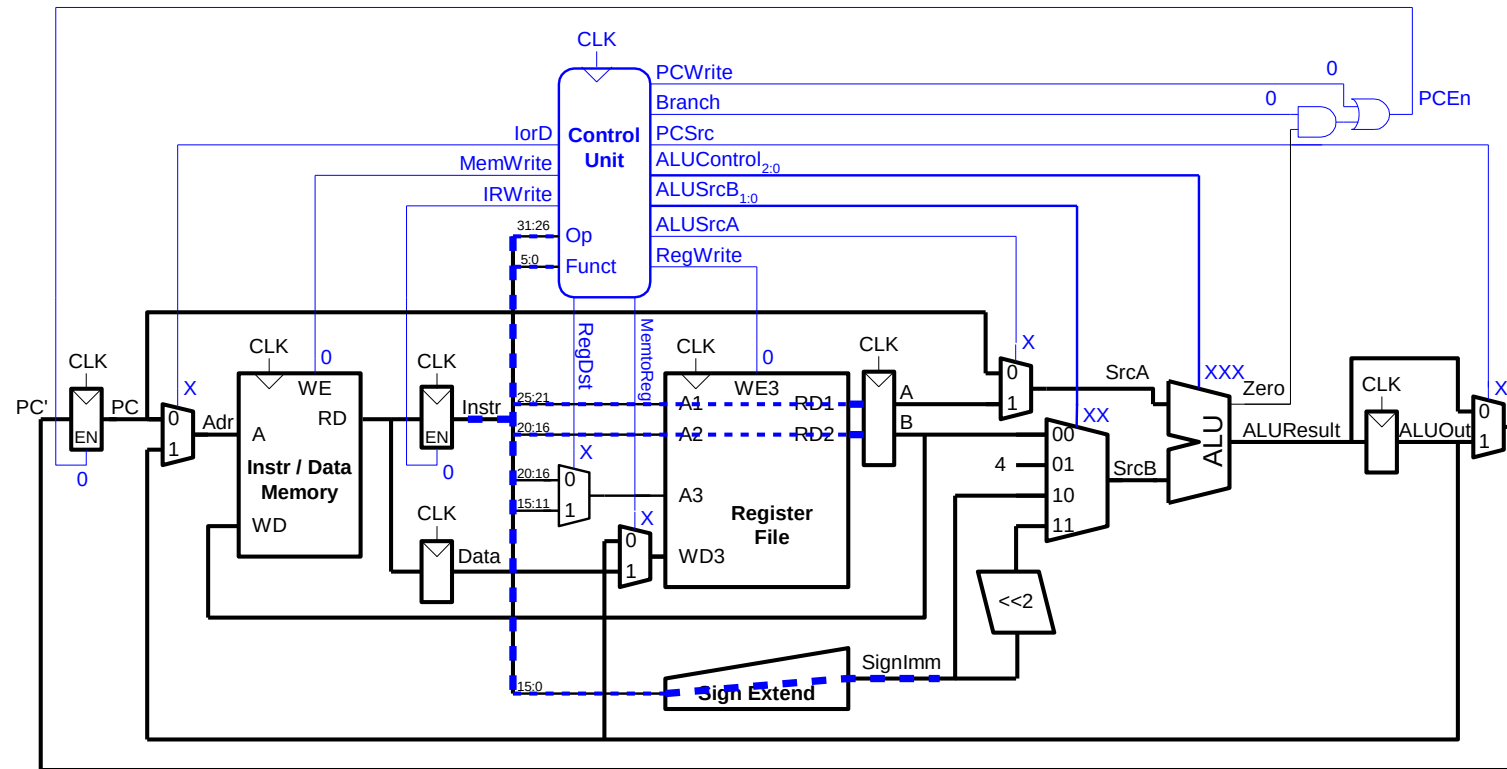
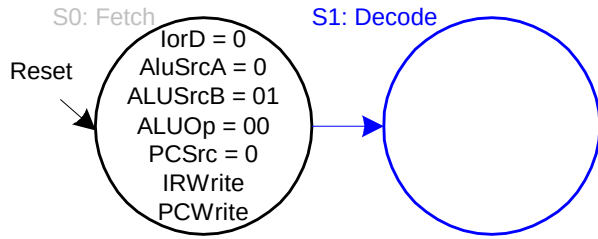
Main Controller FSM: Fetch



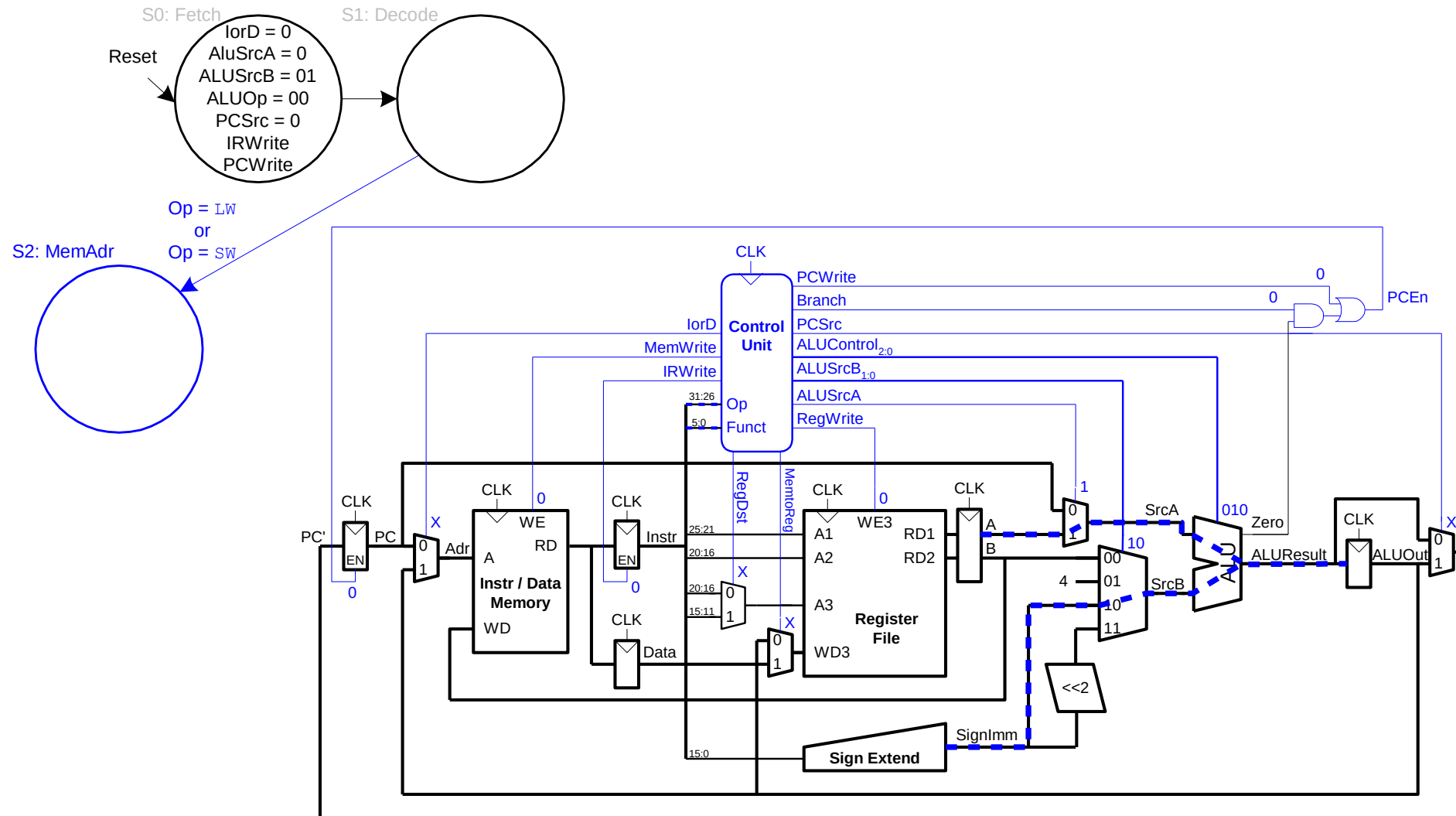
Main Controller FSM: Fetch



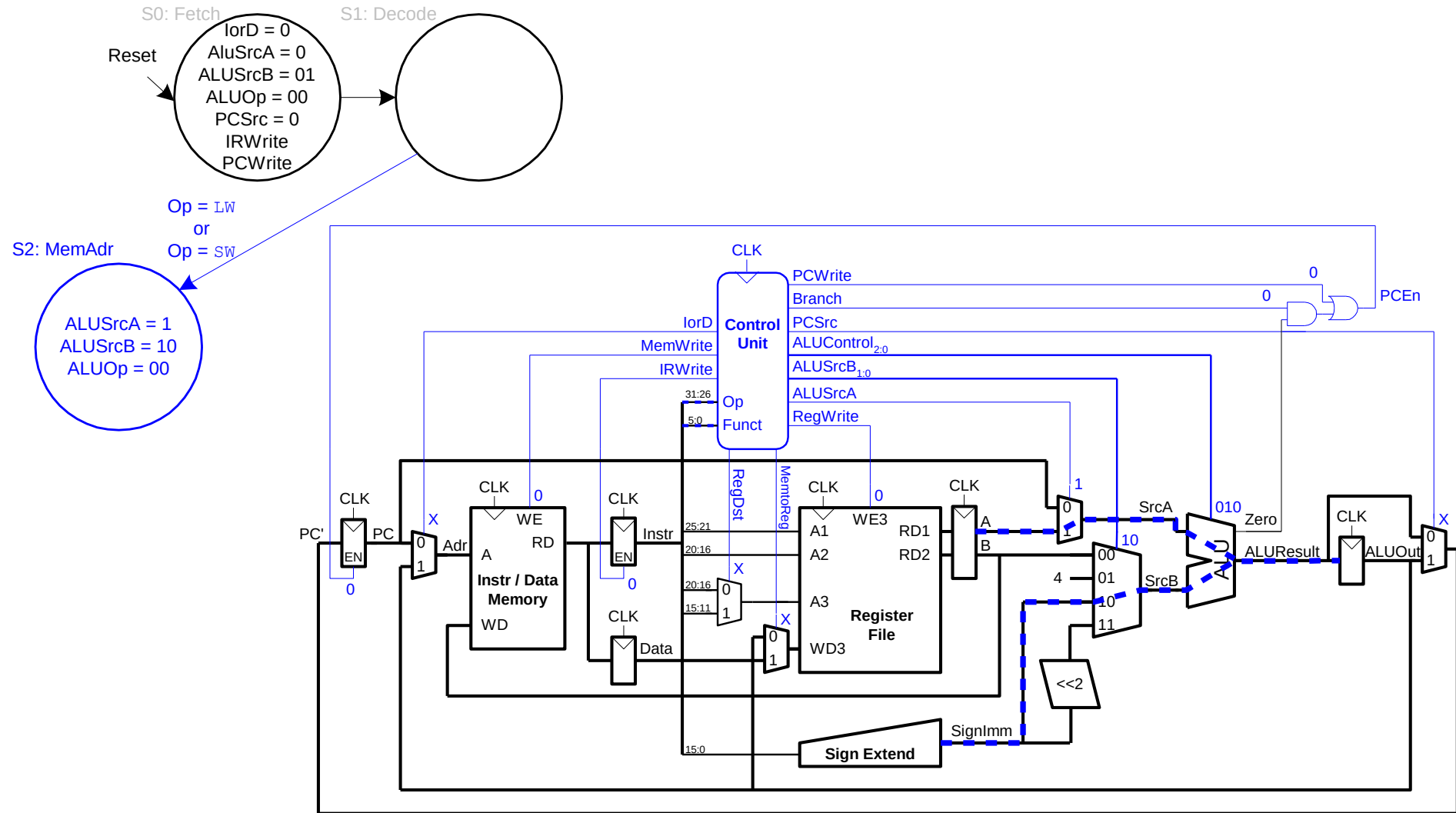
Main Controller FSM: Decode



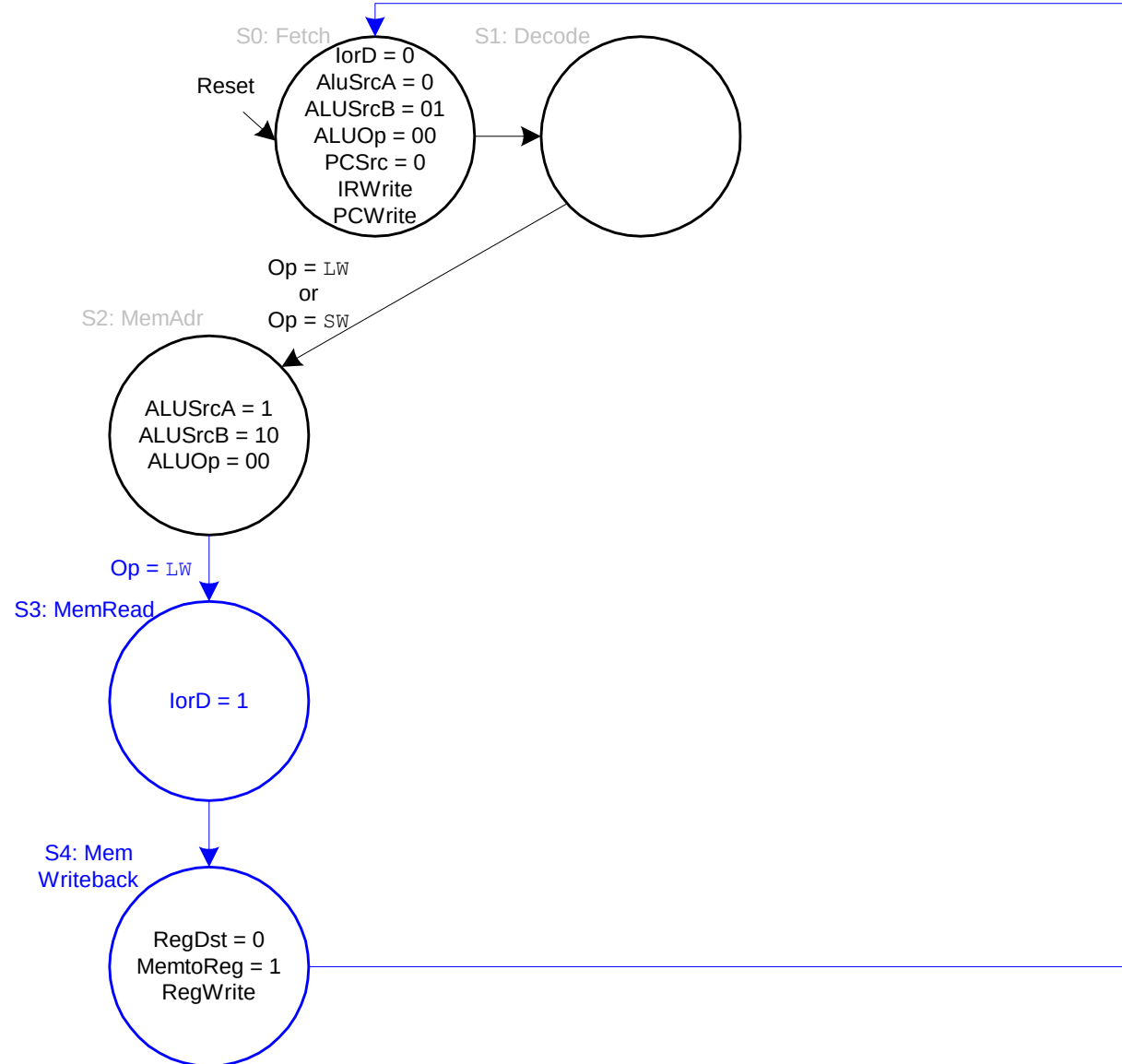
Main Controller FSM: Address Calculation



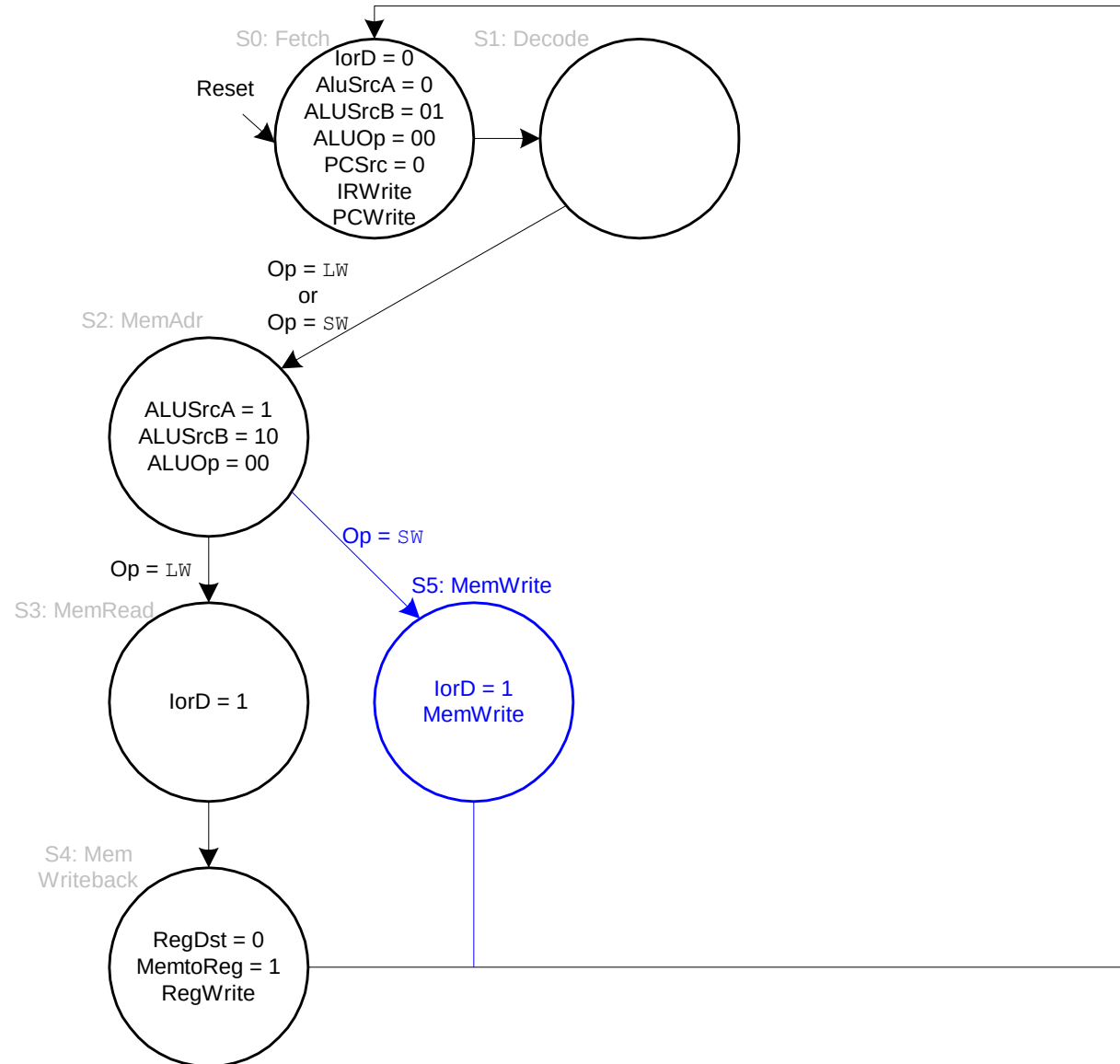
Main Controller FSM: Address Calculation



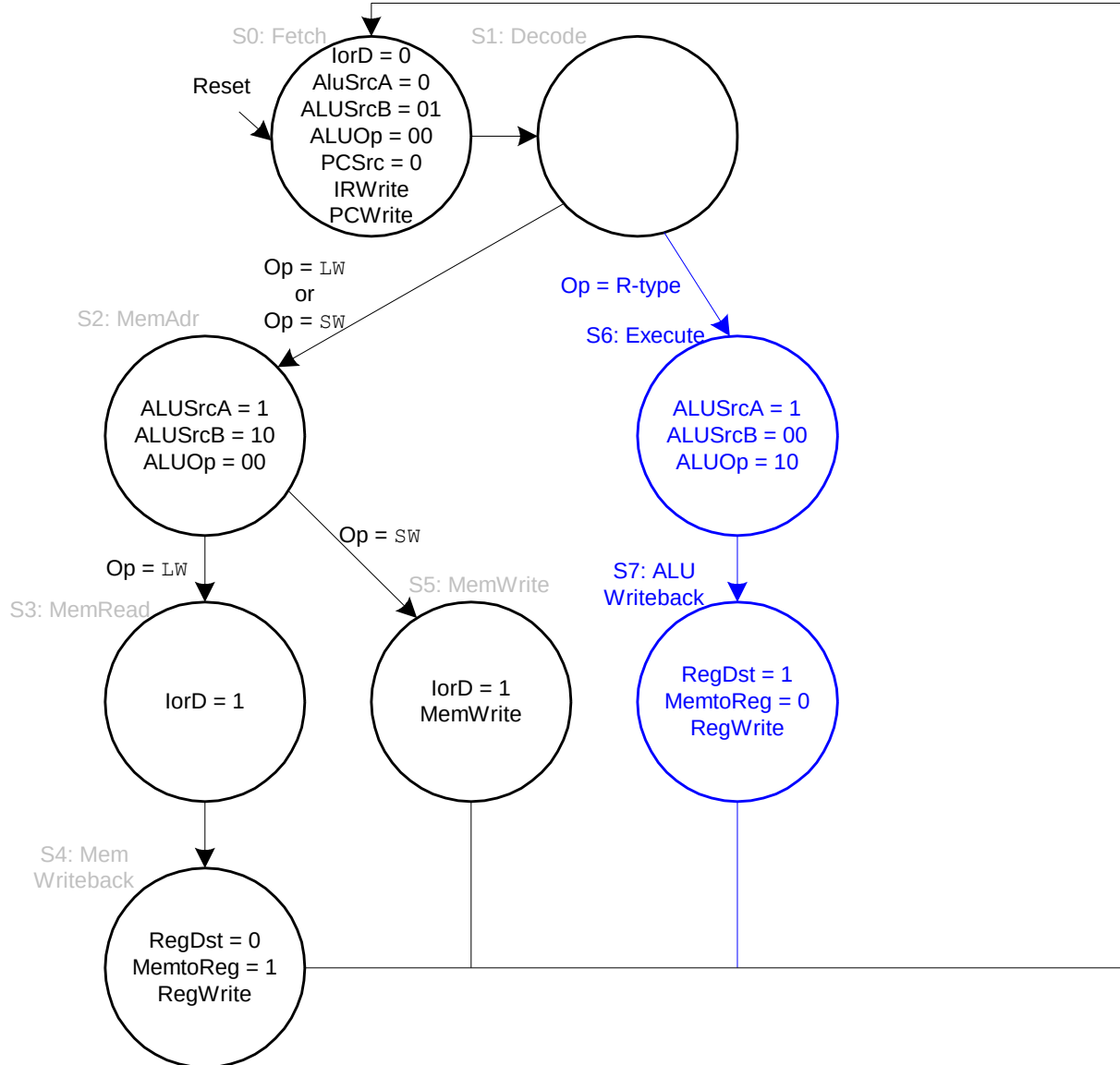
Main Controller FSM: lw



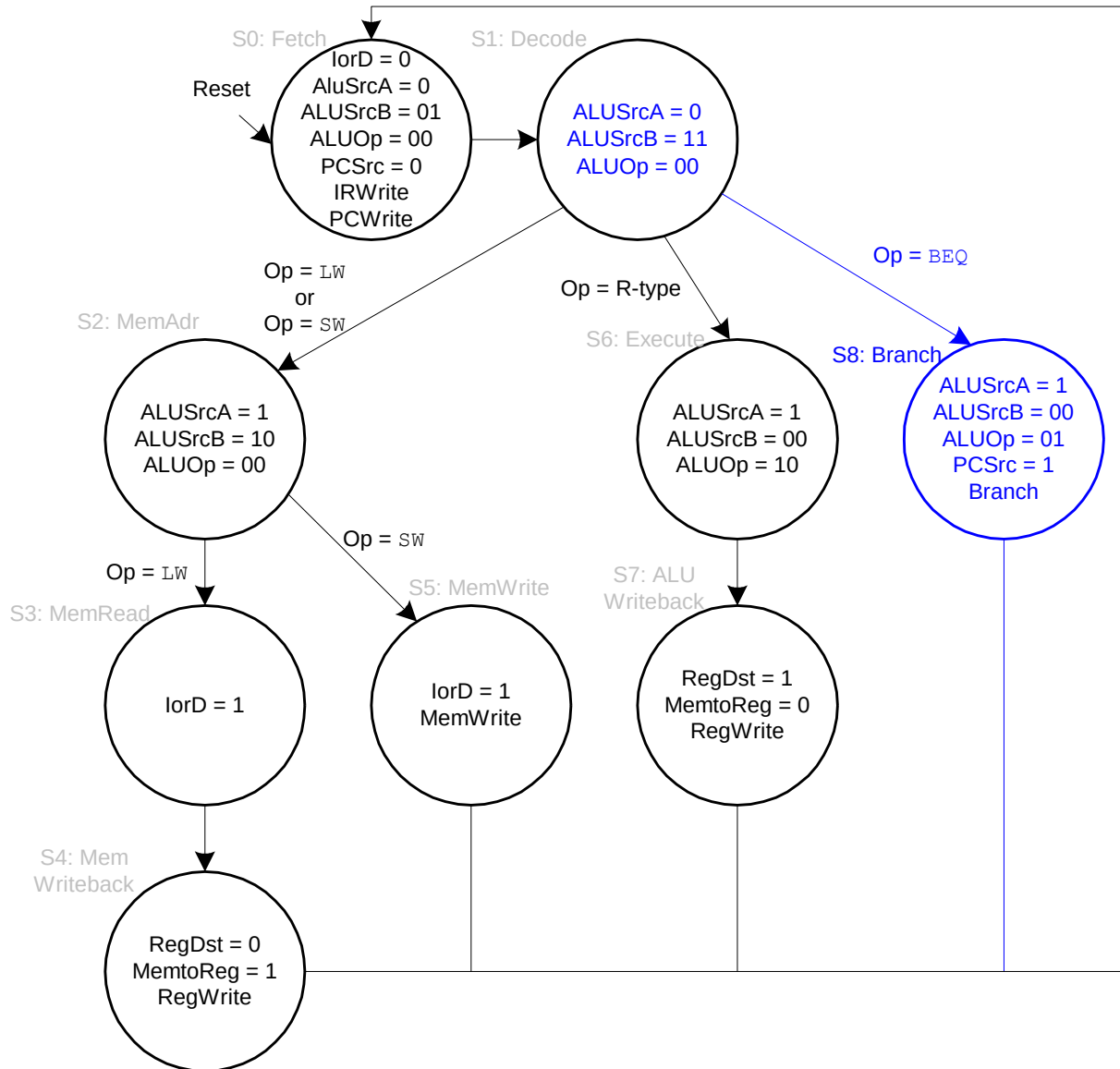
Main Controller FSM: sw



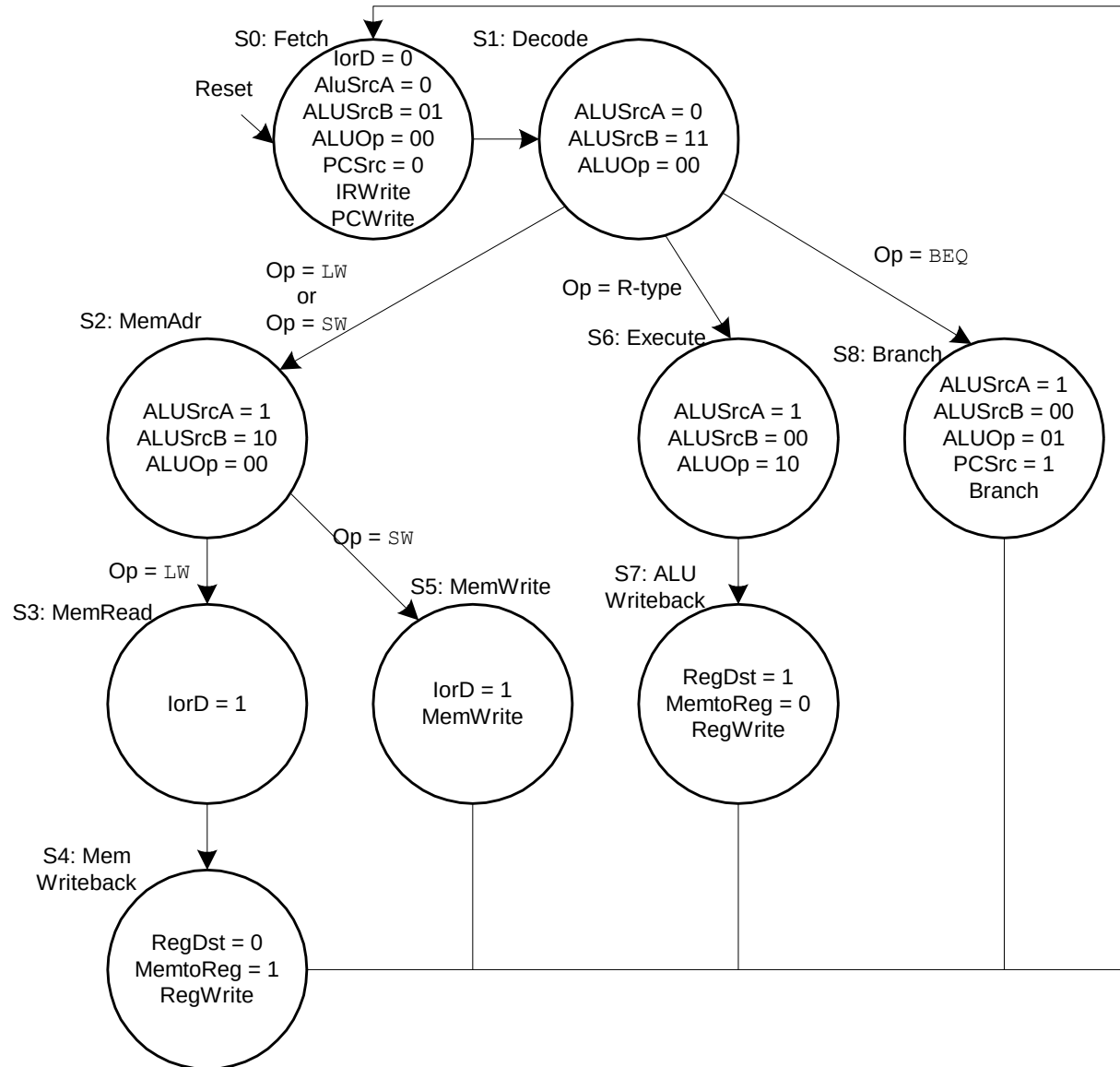
Main Controller FSM: R-Type



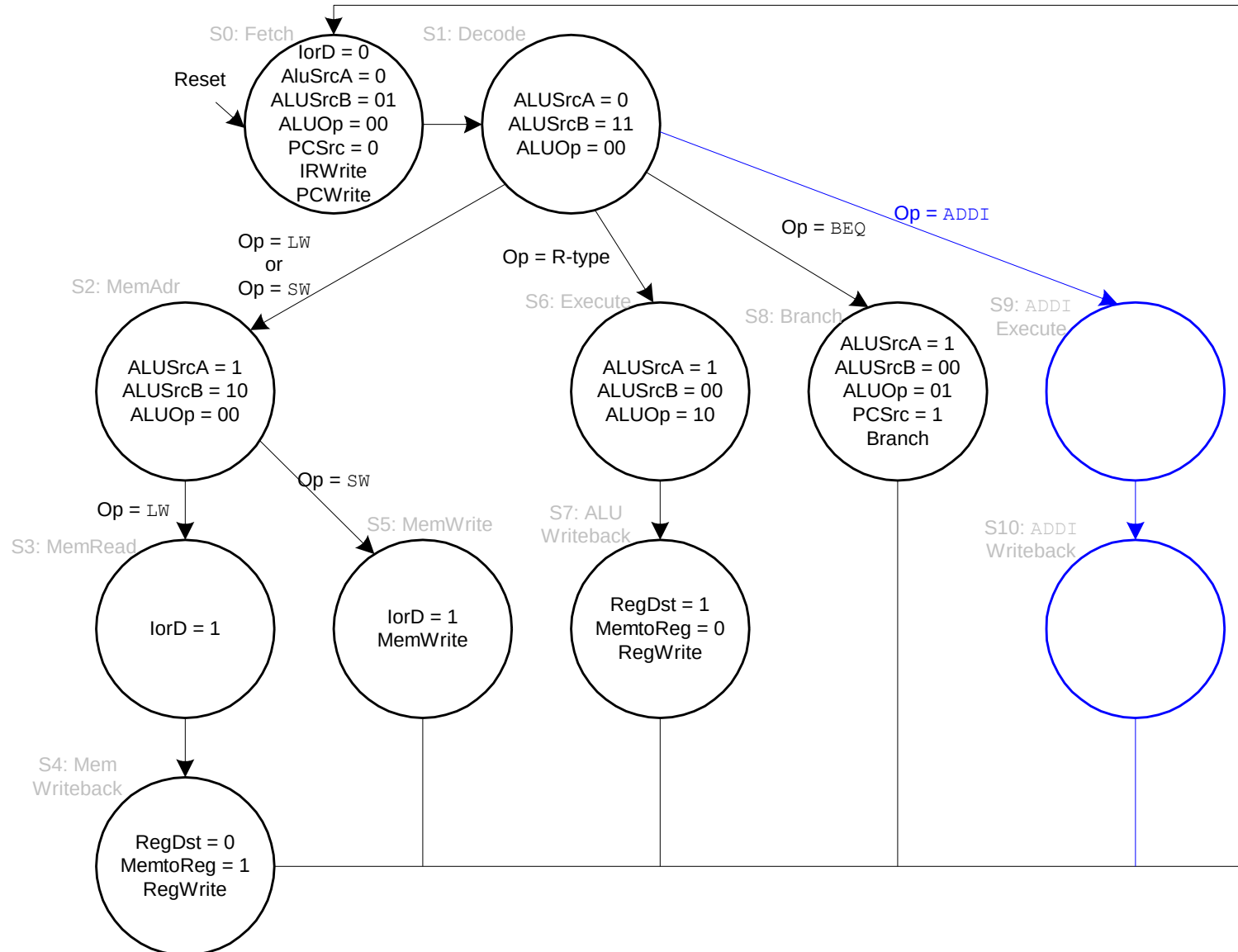
Main Controller FSM: beq



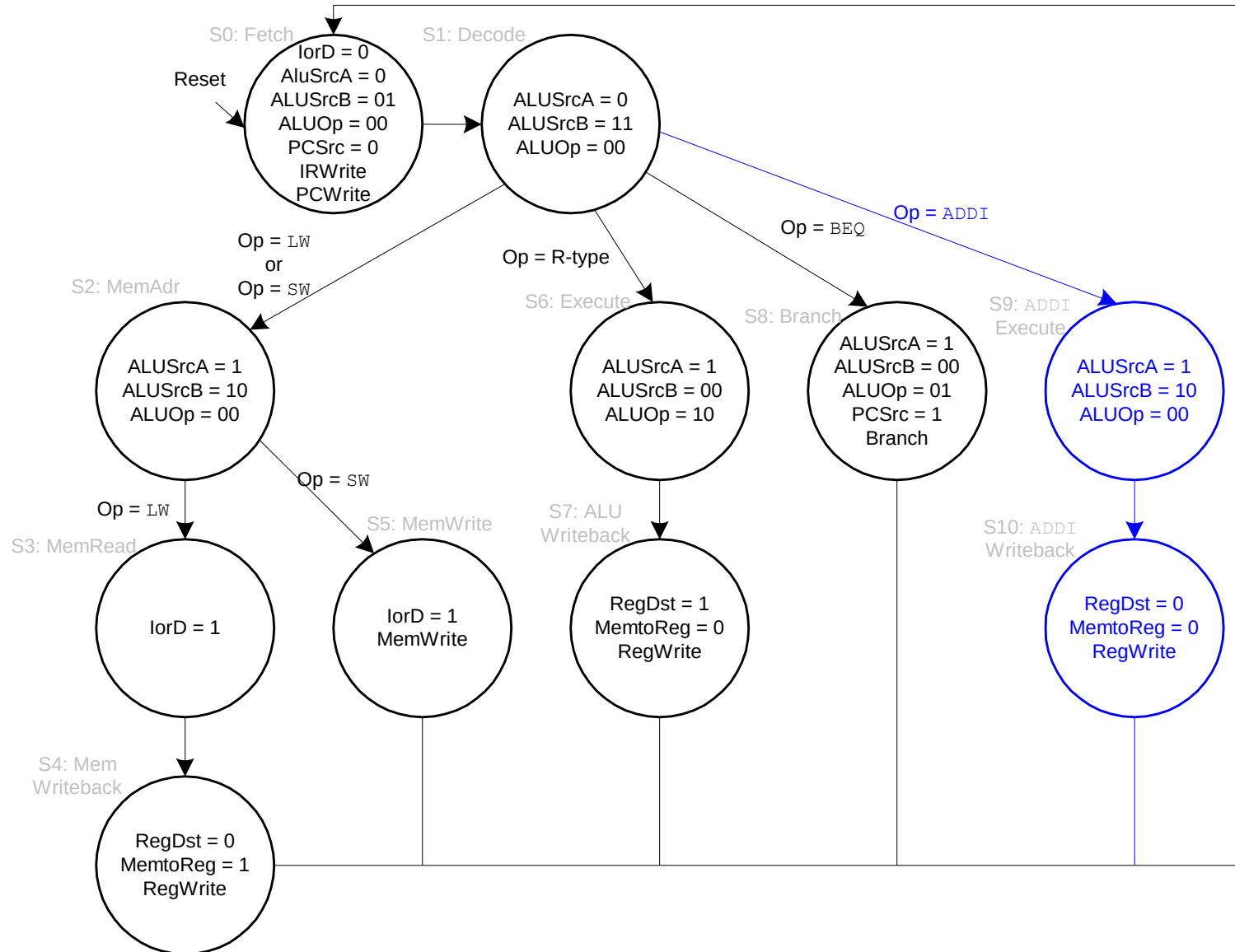
Complete Multi-cycle Controller FSM



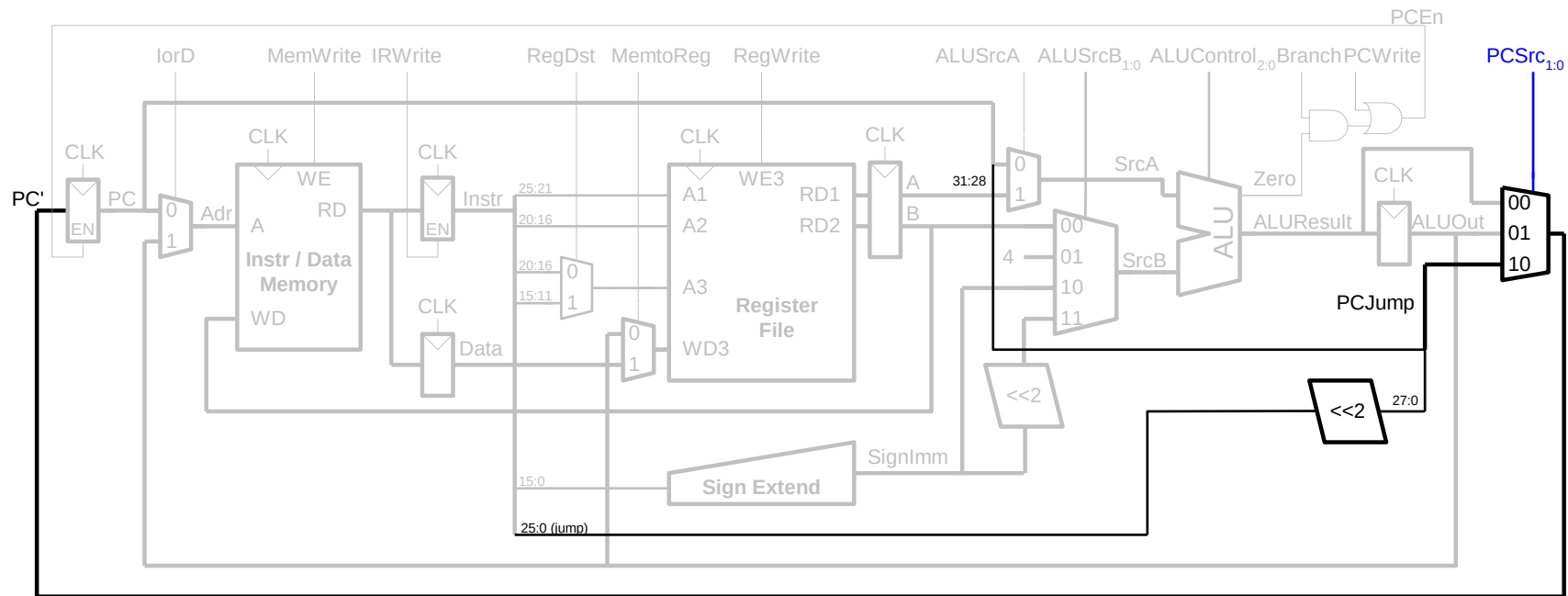
Main Controller FSM: addi



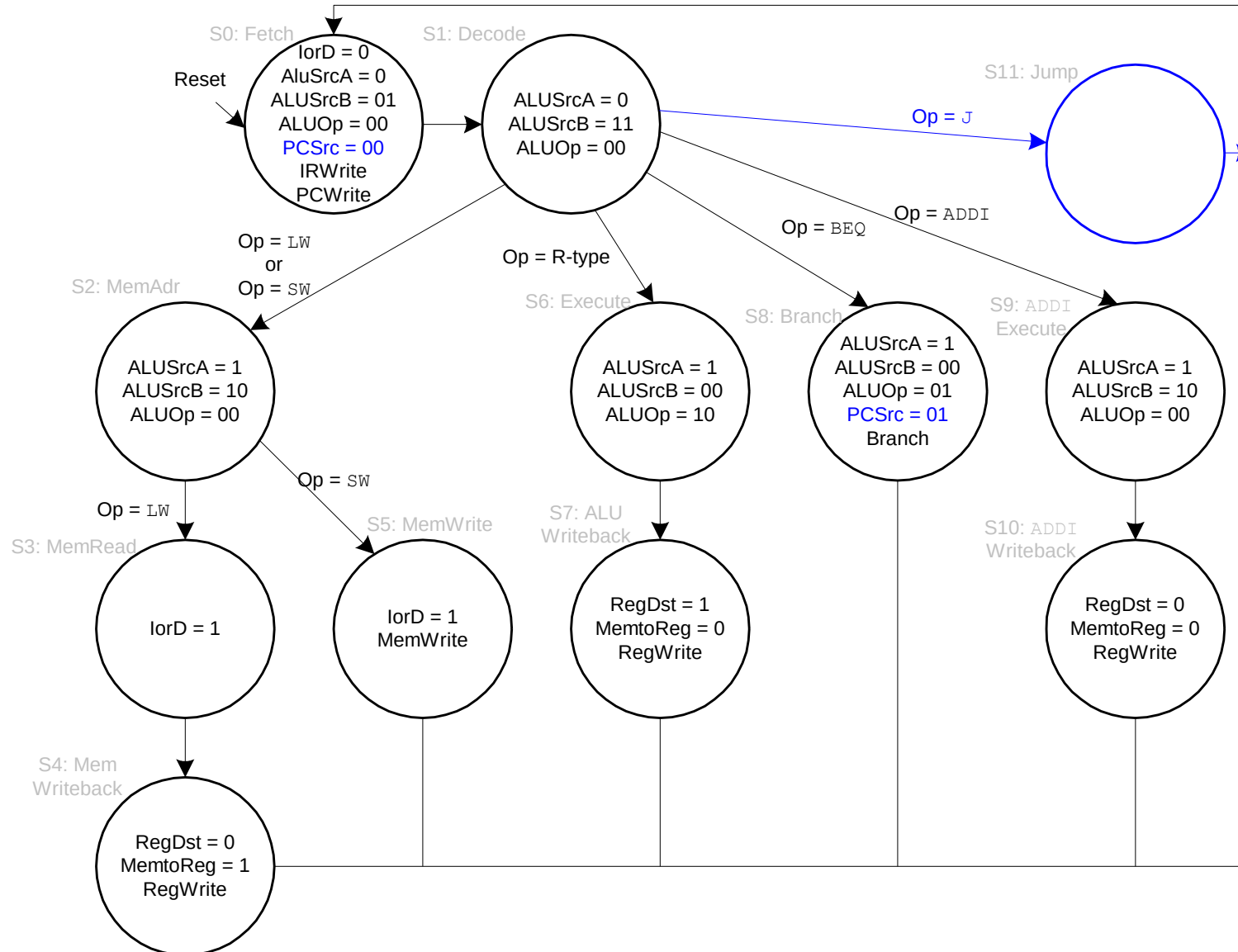
Main Controller FSM: addi



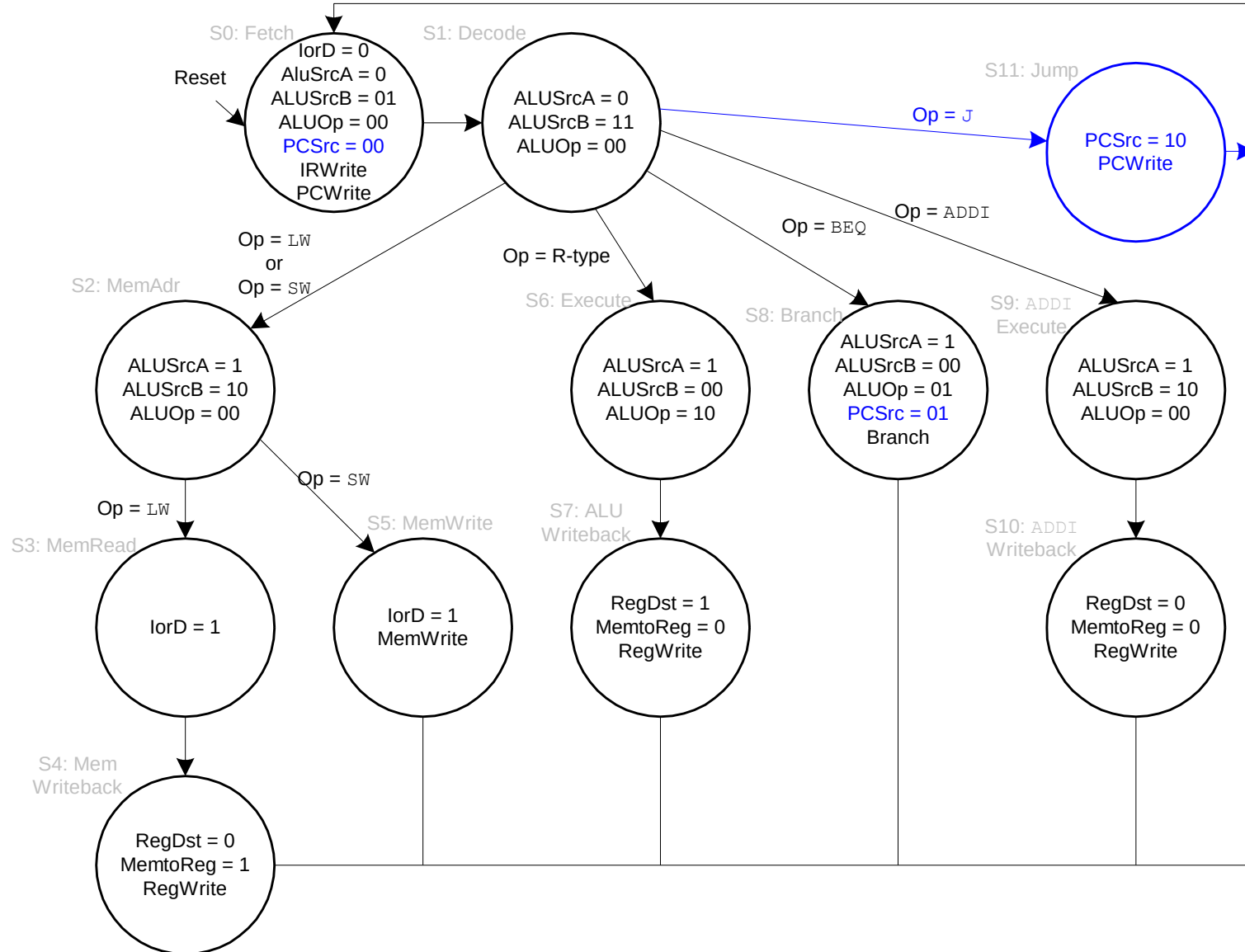
Extended Functionality: j



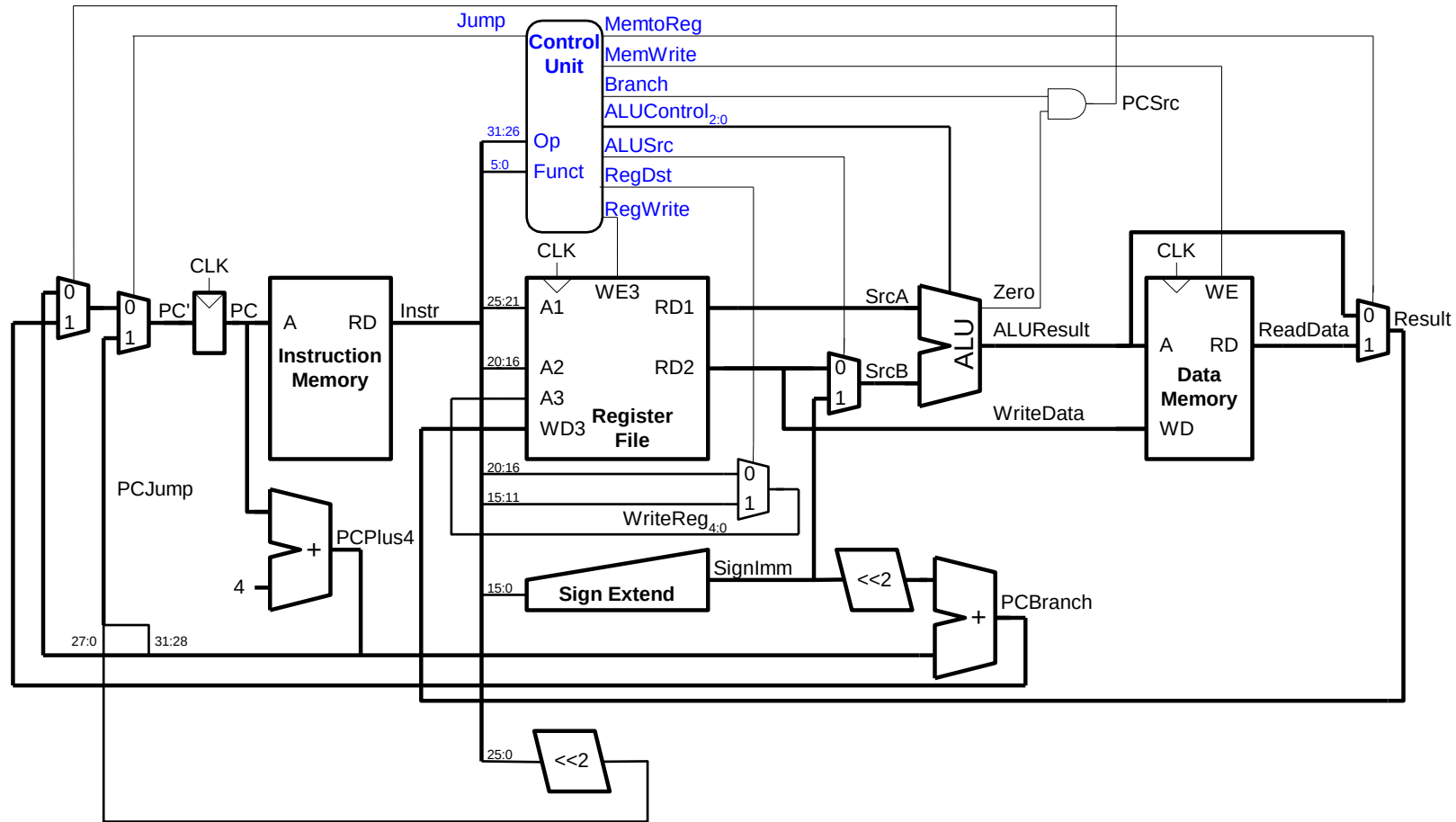
Control FSM: j



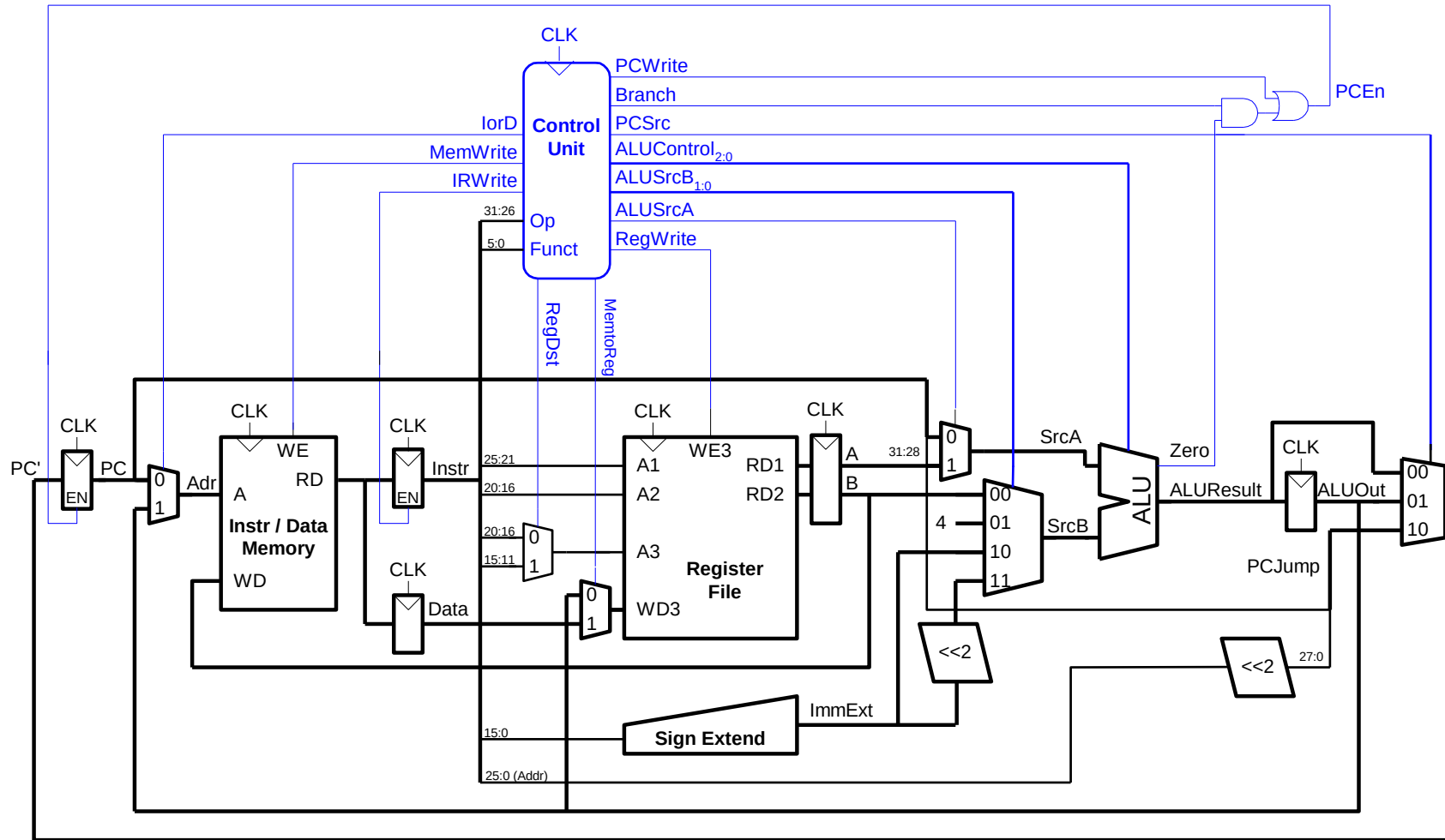
Control FSM: j



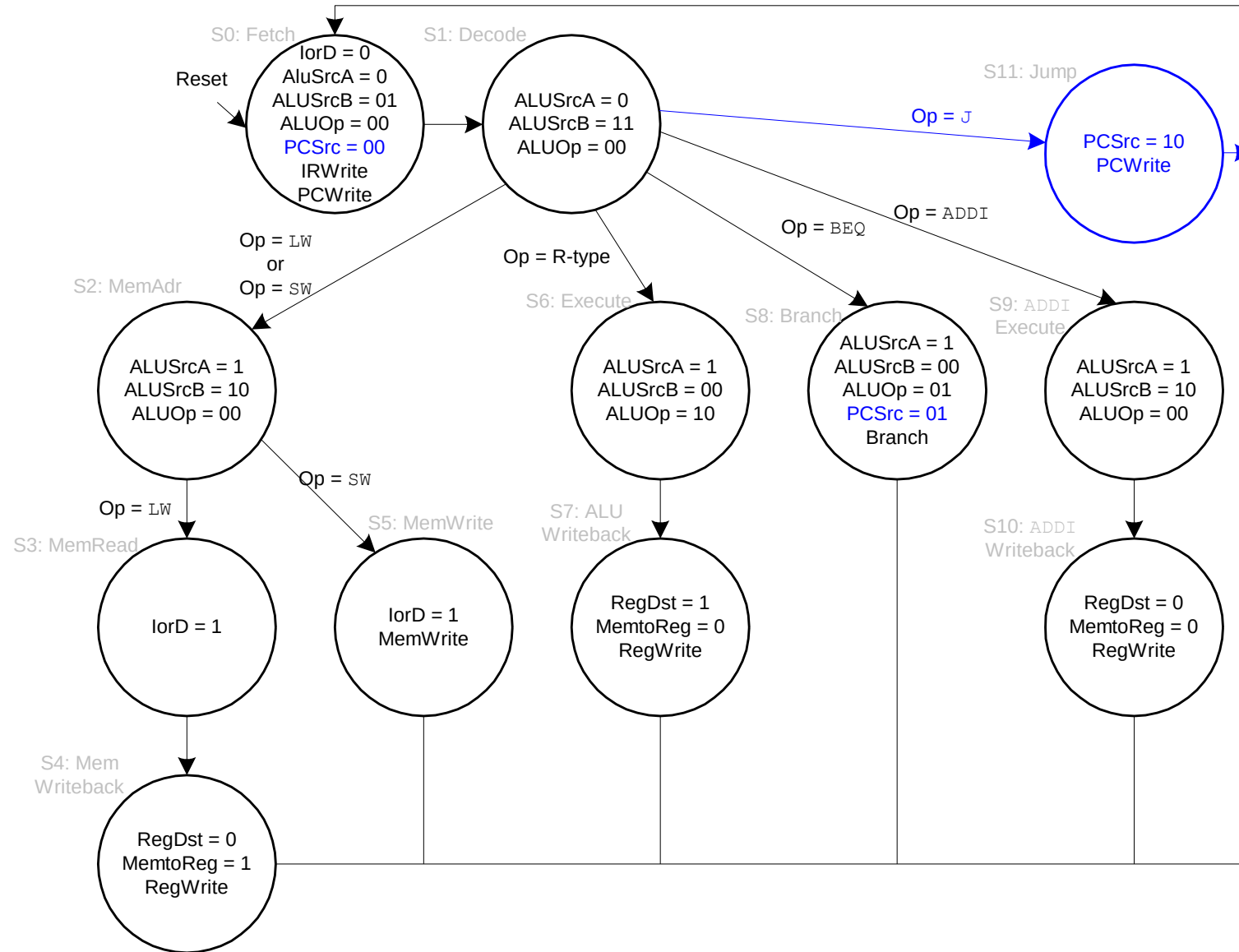
Review: Single-Cycle MIPS Processor



Review: Multi-Cycle MIPS Processor



Review: Multi-Cycle MIPS FSM



**What is the
shortcoming of
this design?**

**What does
this design
assume
about memory?**

What If Memory Takes $>$ One Cycle?

- Stay in the same "memory access" state until memory returns the data
- "Memory Ready?" bit is an input to the control logic that determines the next state

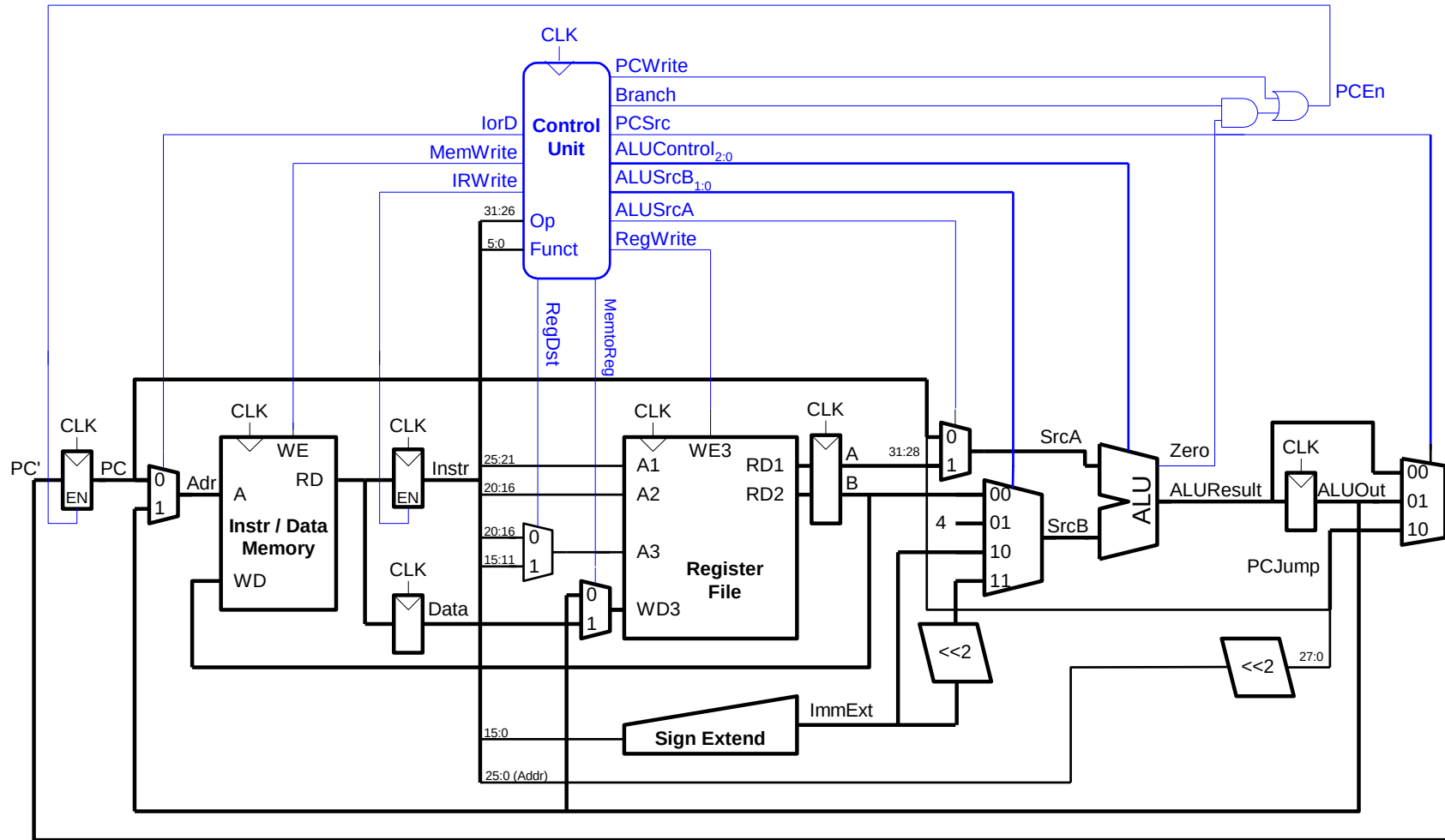
Pipelining

Readings

- Digital Design and Computer Architecture - Sarah Harris, David Harris
 - Chapter 7.5
 - Chapter 7.8.1 – 7.8.3

- Instruction Set Architectures (ISA): LC-3
- Assembly programming: LC-3
- Memory Technologies, Memory Hierarchy
- Caches
- Virtual Memory
- Microarchitecture (principles & single-cycle uarch): MIPS
- Multi-cycle microarchitecture: MIPS
- Pipelining: MIPS
- Issues in Pipelining: Control & Data Dependence Handling, State Maintenance and Recovery, ...
- ...

Review: Multi-Cycle MIPS Processor



Can We Do Better?

- What limitations do you see with the multi-cycle design?
- Limited concurrency
 - Some hardware resources are idle during different phases of instruction processing cycle
 - “Fetch” logic is idle when an instruction is being “decoded” or “executed”
 - Most of the datapath is idle when a memory access is happening

Can We Use the Idle Hardware to Improve Concurrency?

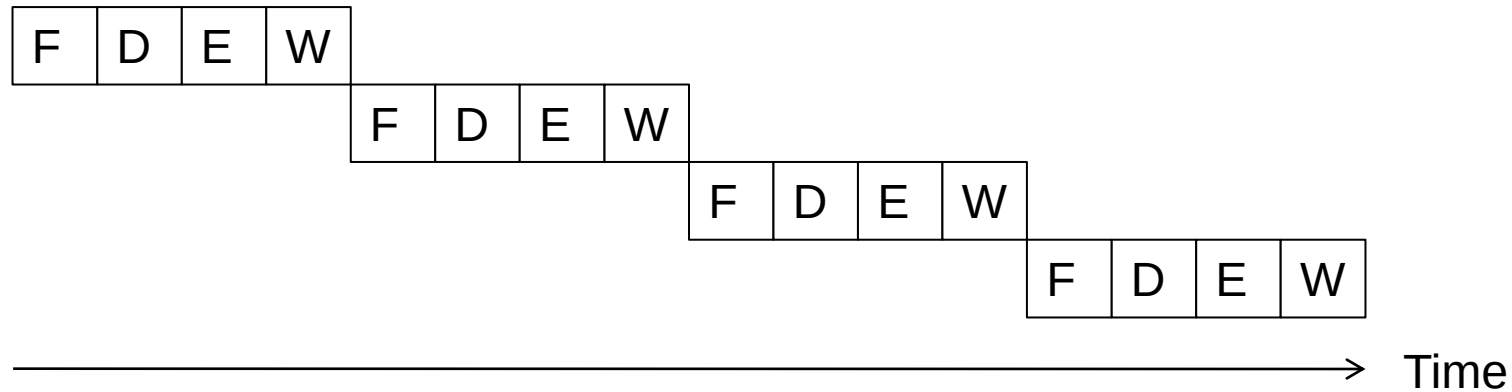
- Goal: **More concurrency → Higher instruction throughput** (i.e., more “work” completed in one cycle)
- Idea: When an instruction is using some resources in its processing phase, **process other instructions on idle resources** not needed by that instruction
 - E.g., when an instruction is being decoded, fetch the next instruction
 - E.g., when an instruction is being executed, decode another instruction
 - E.g., when an instruction is accessing data memory (ld/st), execute the next instruction
 - E.g., when an instruction is writing its result into the register file, access memory for the next instruction

Pipelining: Basic Idea

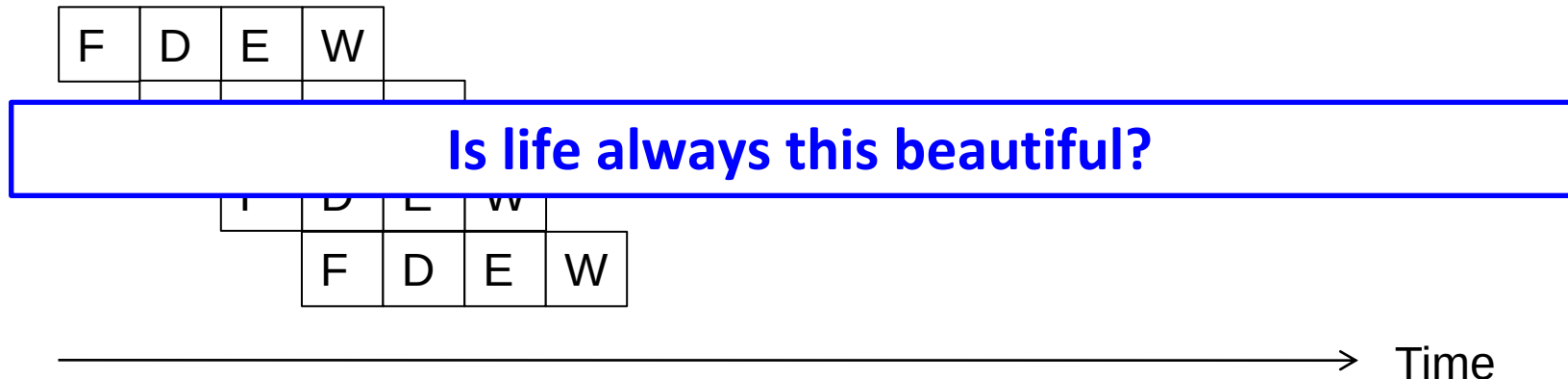
- More systematically:
 - Pipeline the execution of multiple instructions
 - Analogy: "Assembly line processing" of instructions
- Idea:
 - Divide the instruction processing cycle into distinct "stages" of processing
 - Ensure there are enough hardware resources to process one instruction in each stage
 - Process a **different** instruction in each stage
 - Instructions consecutive in program order are processed in consecutive stages
- Benefit: Increases instruction processing throughput (1/CPI)
- Downside: Start thinking about this...

Example: Execution of Four Independent ADDs

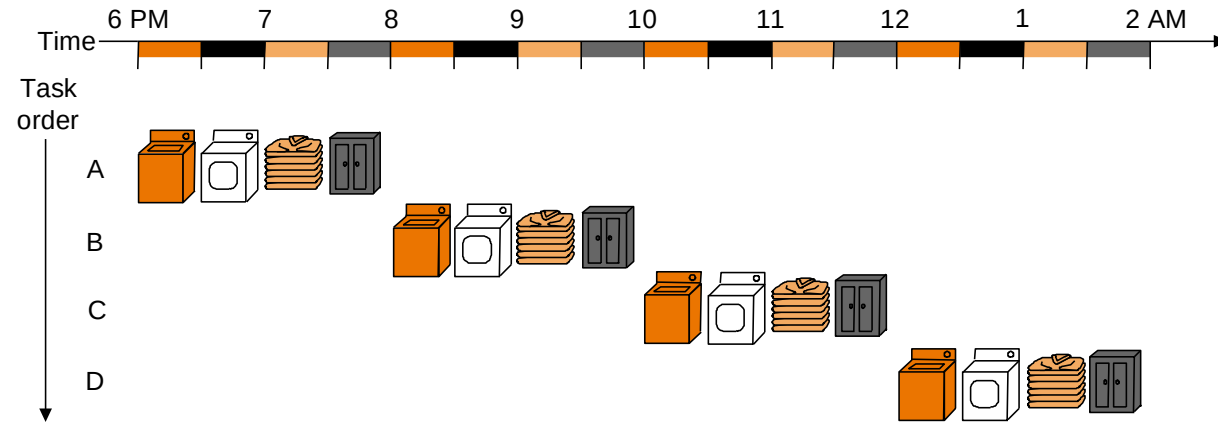
- Multi-cycle: 4 cycles per instruction



- Pipelined: 4 cycles per 4 instructions (steady state)

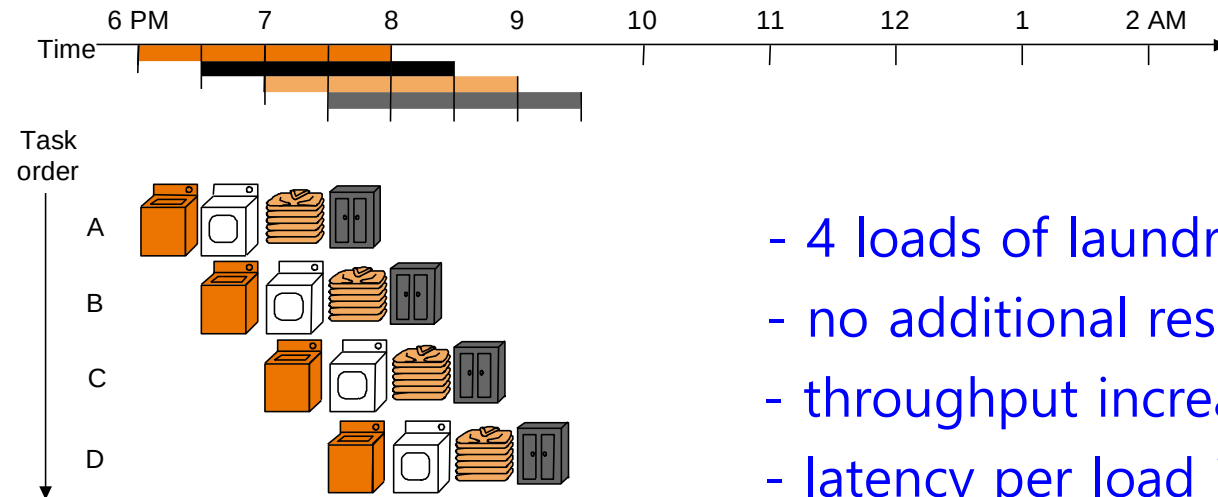
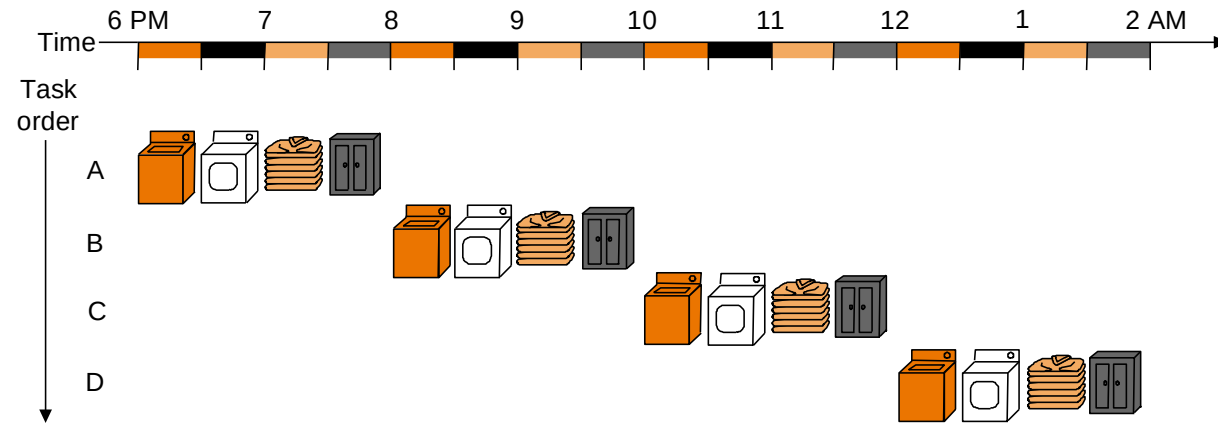


The Laundry Analogy



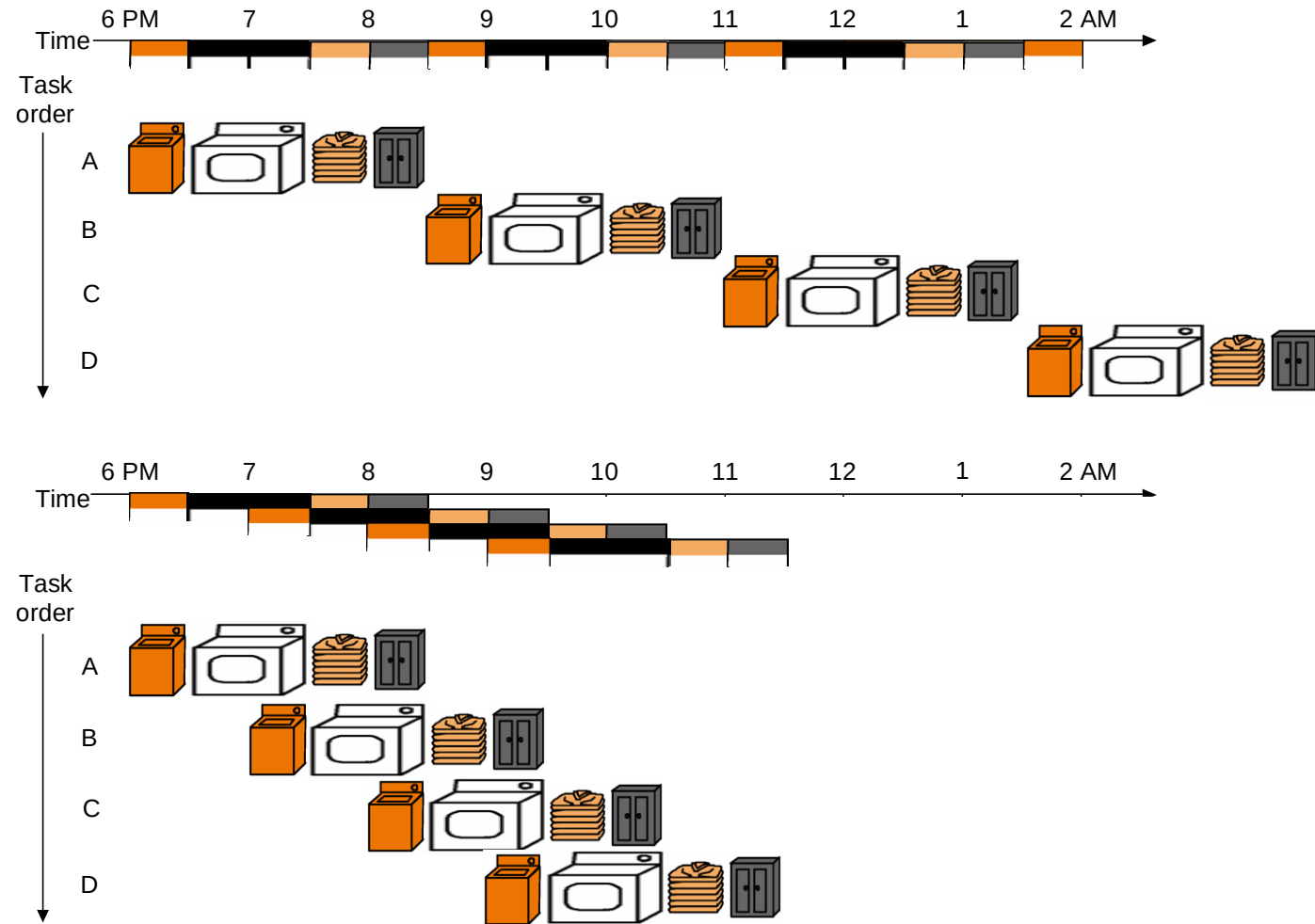
- “place one dirty load of clothes in the washer”
 - “when the washer is finished, place the wet load in the dryer”
 - “when the dryer is finished, take out the dry load and fold”
 - “when folding is finished, ask your roommate (??) to put the clothes away”
- steps to do a load are sequentially dependent
 - no dependence between different loads
 - different steps do not share resources

Pipelining Multiple Loads of Laundry



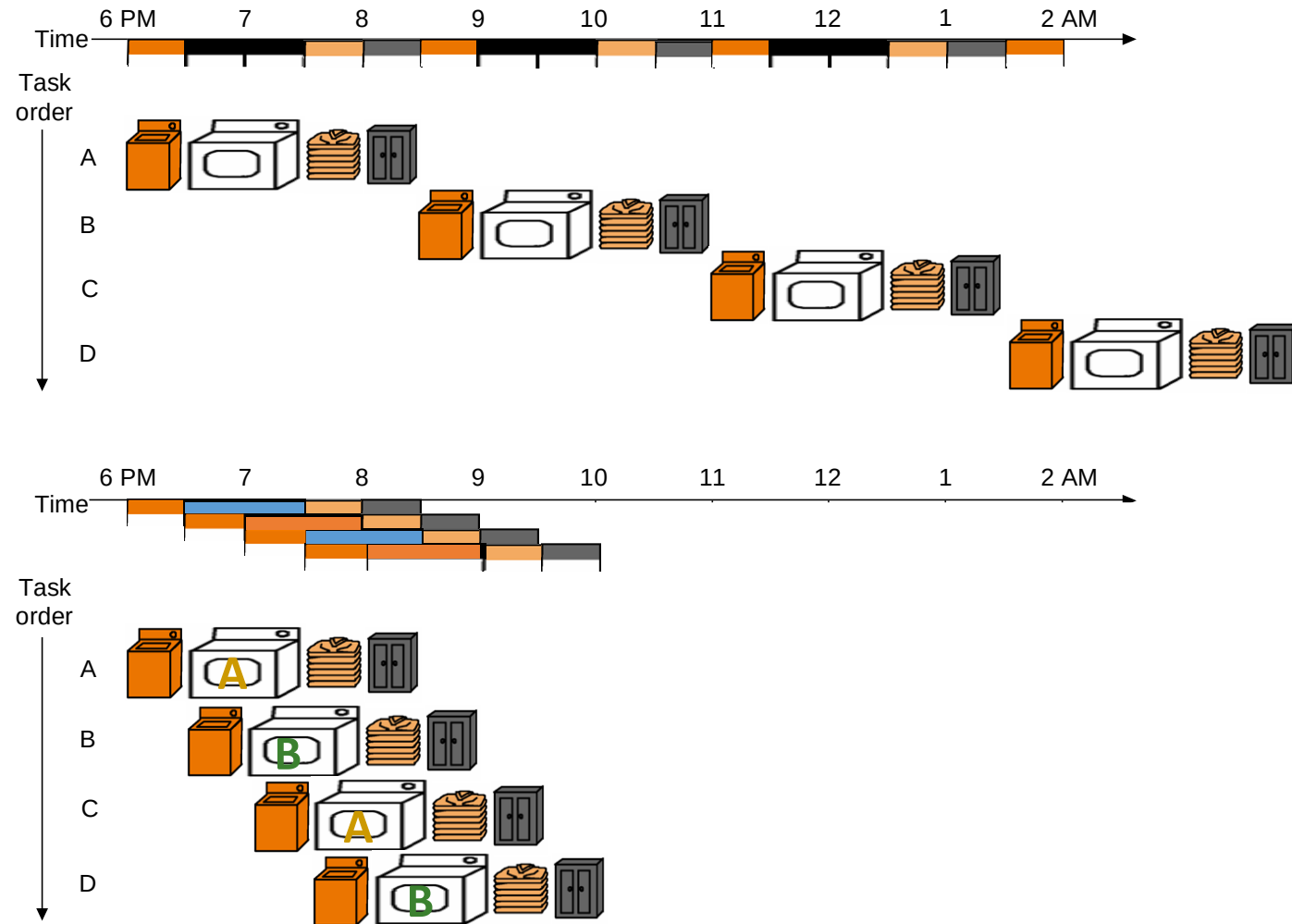
- 4 loads of laundry in parallel
- no additional resources
- throughput increased by 4
- latency per load is the same

Pipelining Multiple Loads of Laundry: In Practice



the slowest step decides throughput

Pipelining Multiple Loads of Laundry: In Practice



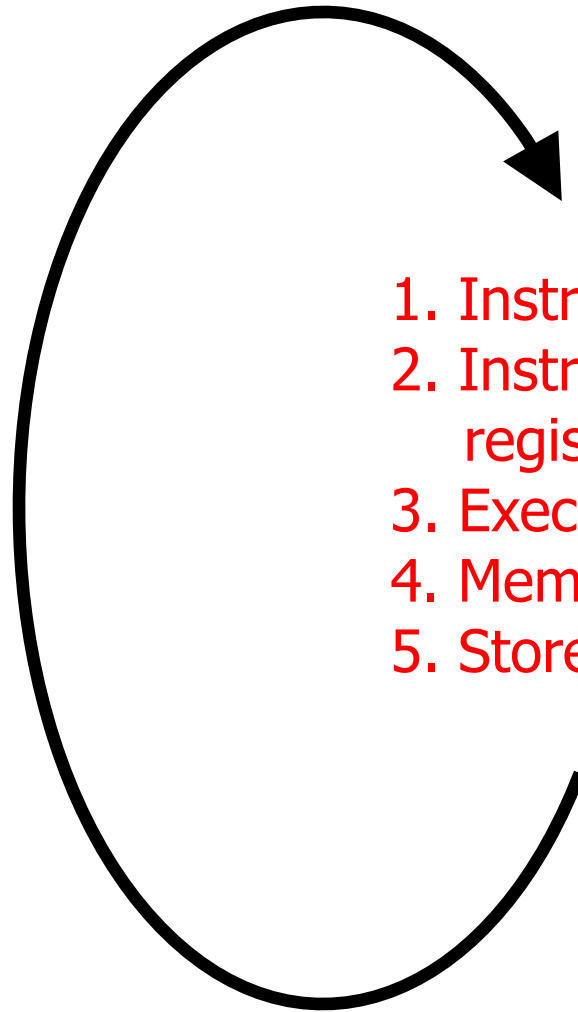
throughput restored (2 loads per hour) using 2 dryers

An Ideal Pipeline

- Goal: **Increase throughput with little increase in cost** (hardware cost, in case of instruction processing)
- Repetition of **identical operations**
 - The same operation is repeated on a large number of different inputs (e.g., all laundry loads go through the same steps)
- Repetition of **independent operations**
 - No dependencies between repeated operations
- **Uniformly partitionable suboperations**
 - Processing can be evenly divided into uniform-latency suboperations (that do not share resources)
- Fitting examples: automobile assembly line, doing laundry
 - What about the instruction processing "cycle"?

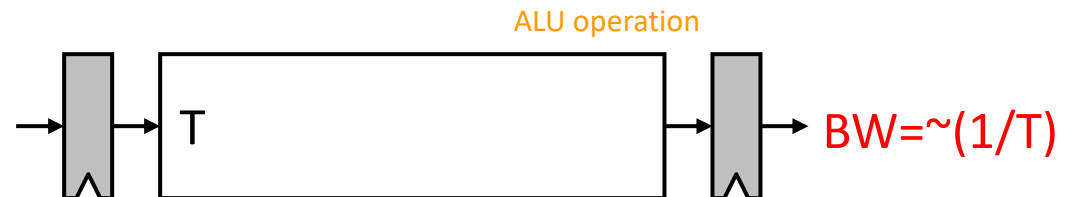
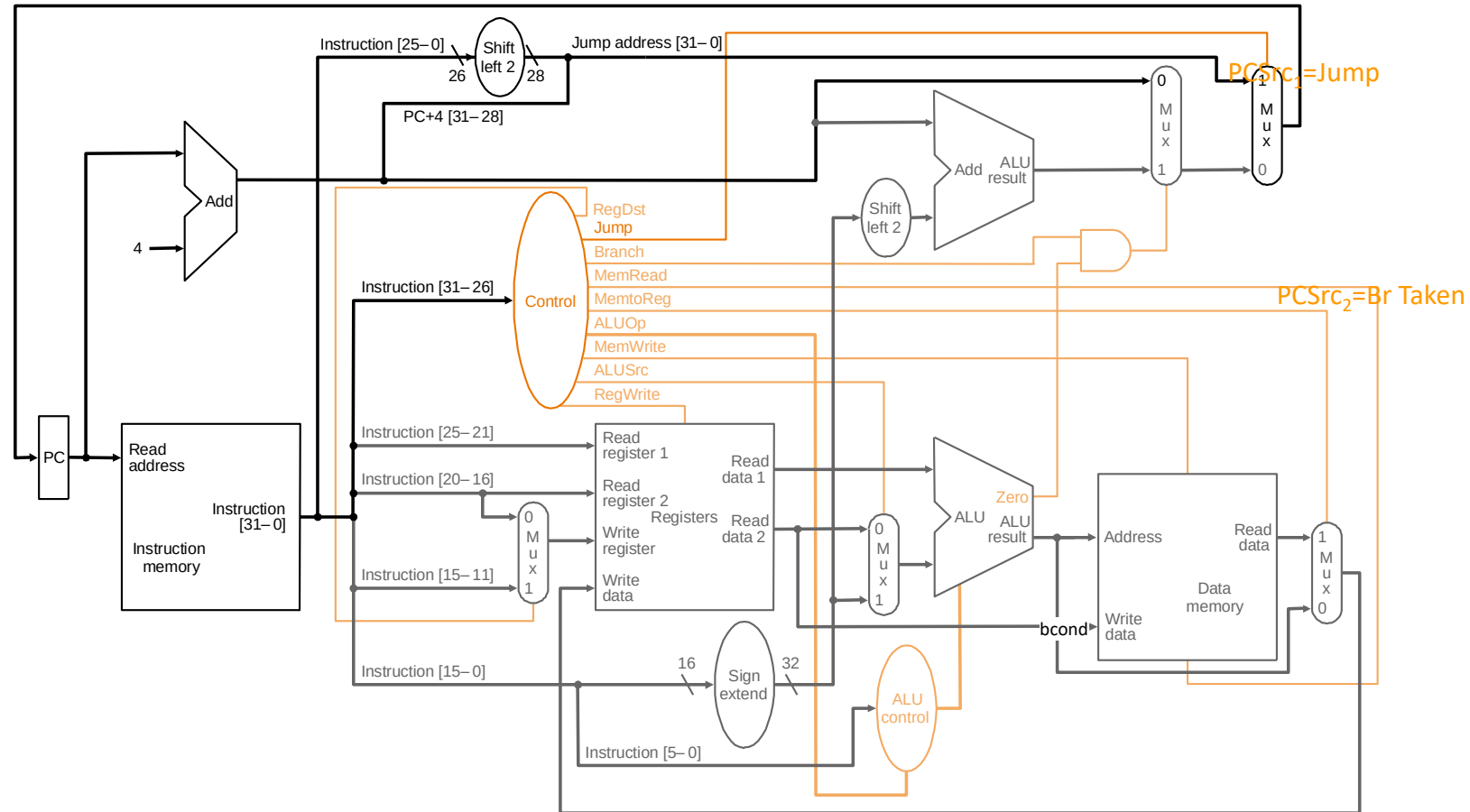
Pipelining Instruction Processing

Remember: The Instruction Processing Cycle

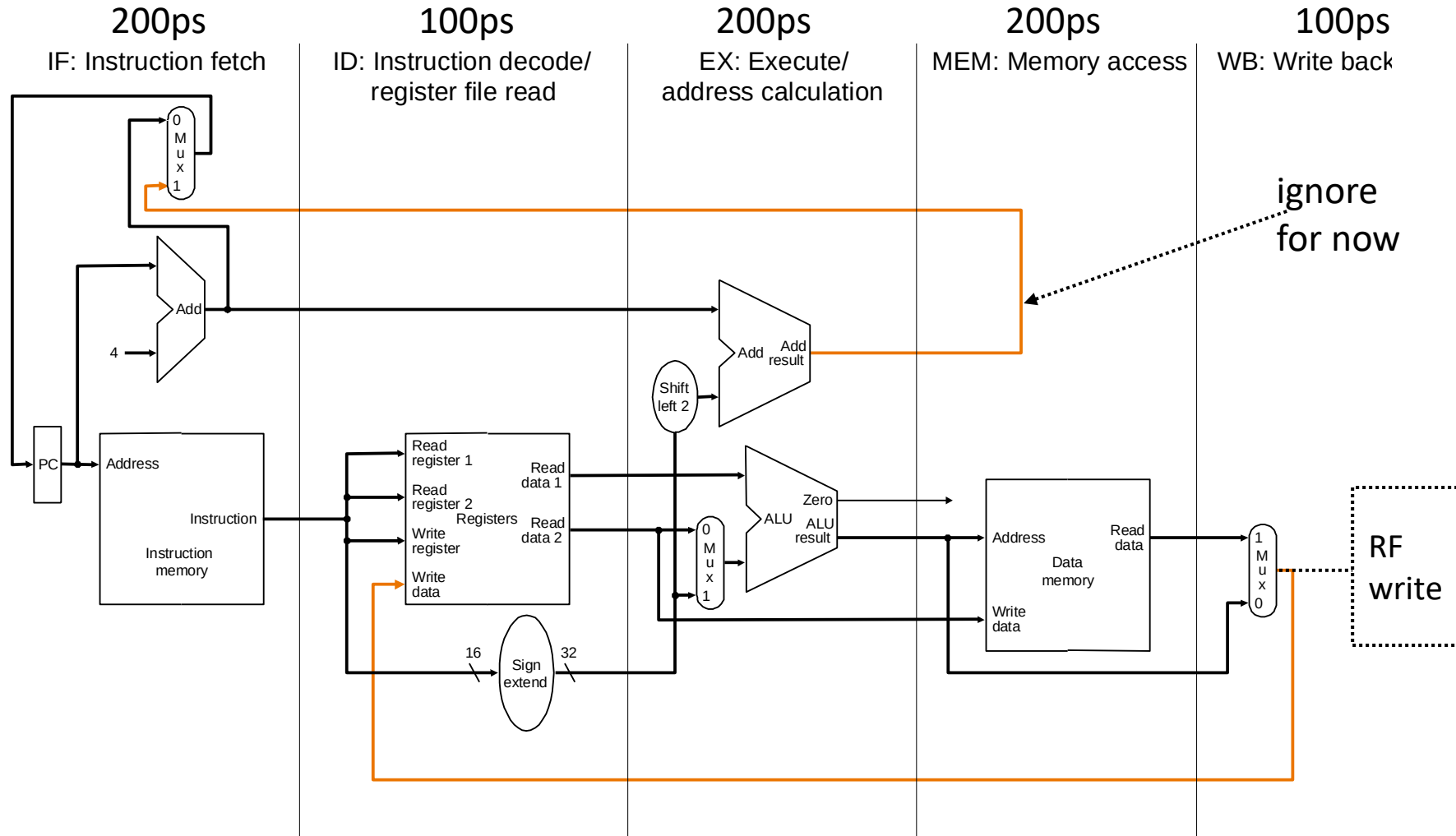


1. Instruction fetch (IF)
2. Instruction decode and register operand fetch (ID/RF)
3. Execute/Evaluate memory address (EX/AG)
4. Memory operand fetch (MEM)
5. Store/writeback result (WB)

Remember the Single-Cycle Uarch



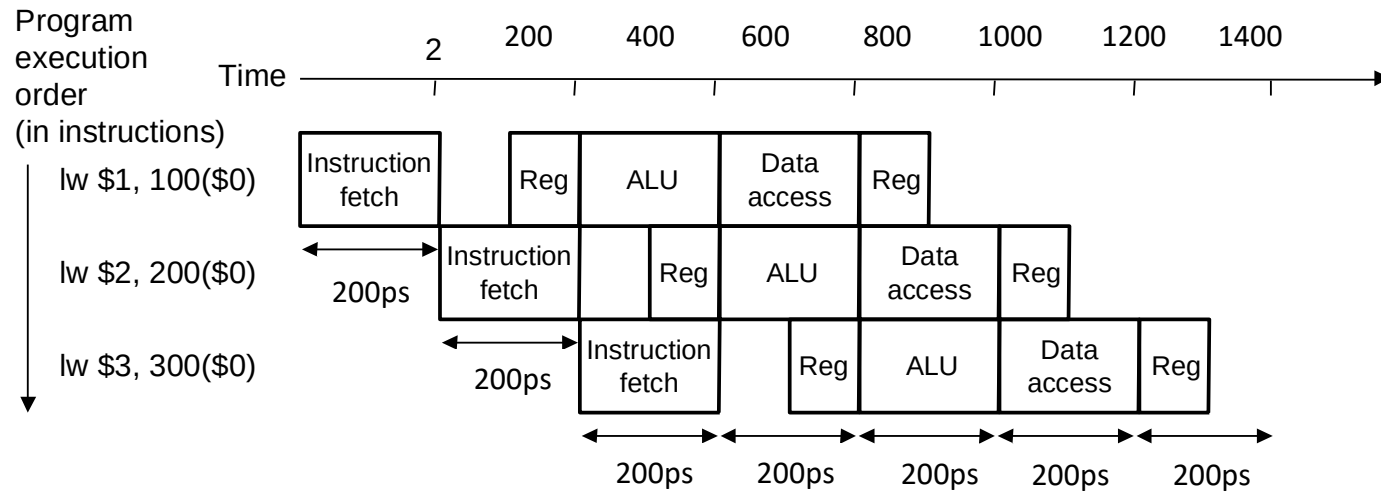
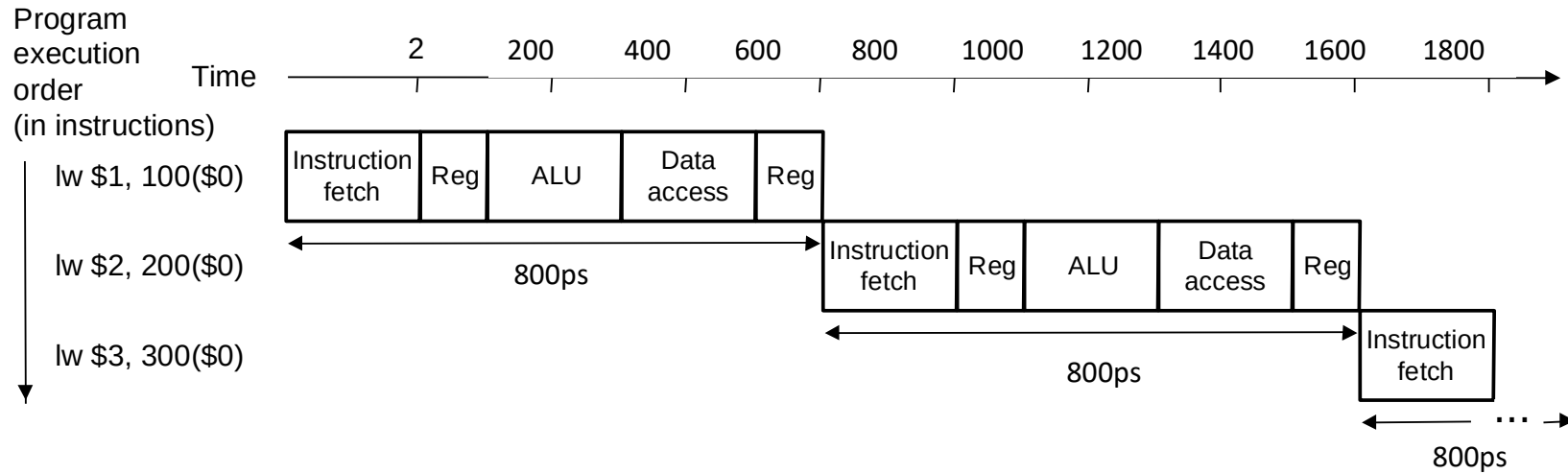
Dividing Into Stages



Is this the correct partitioning?

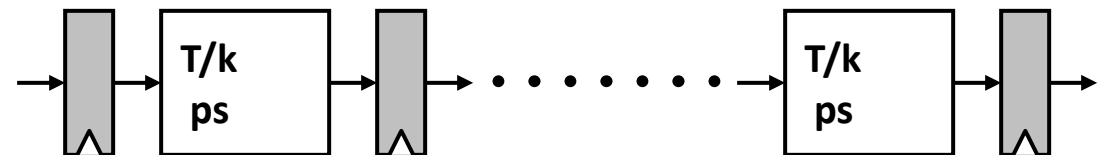
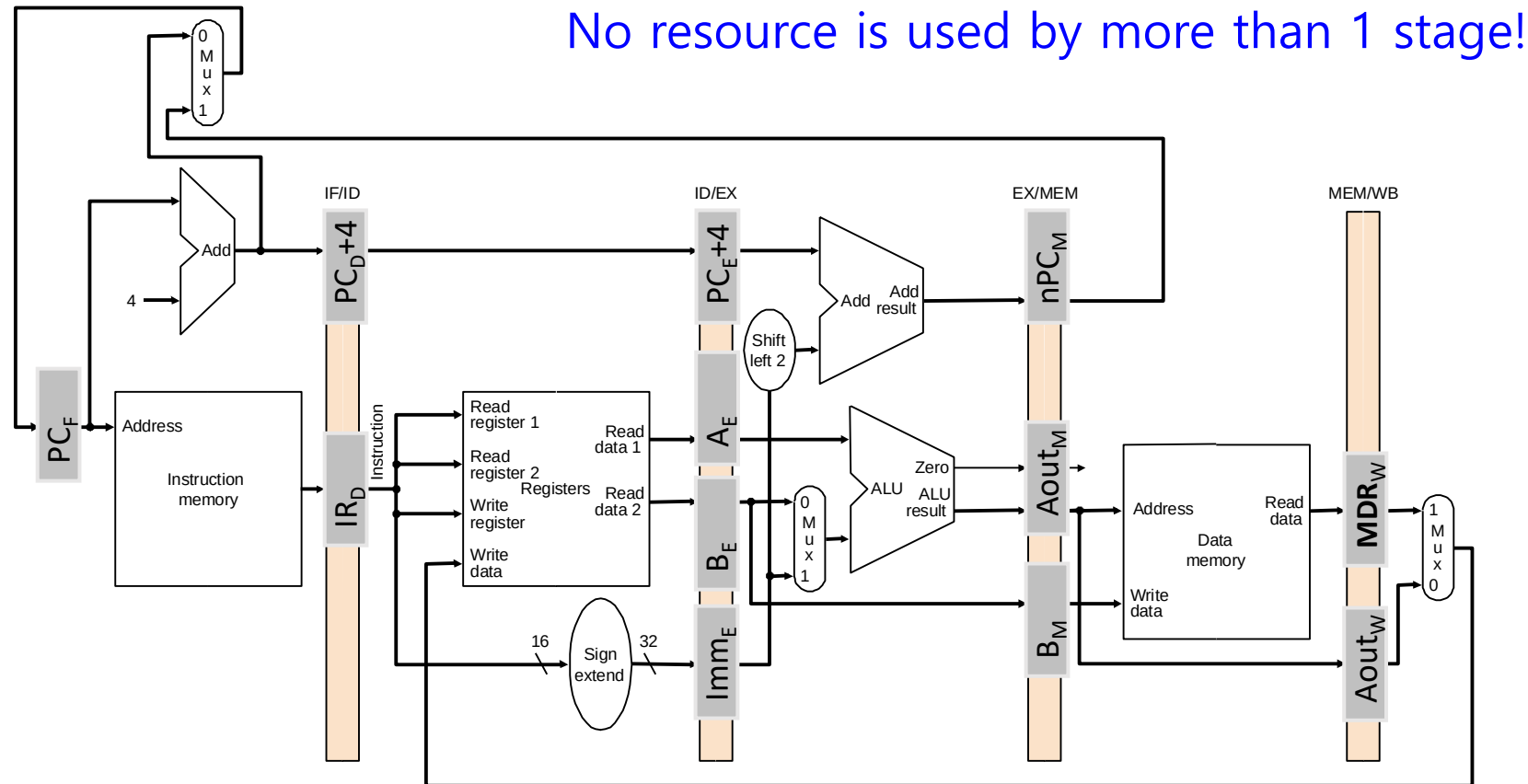
Why not 4 or 6 stages? Why not different boundaries?

Instruction Pipeline Throughput



5-stage speedup is 4, not 5 as predicted by the ideal model. Why?

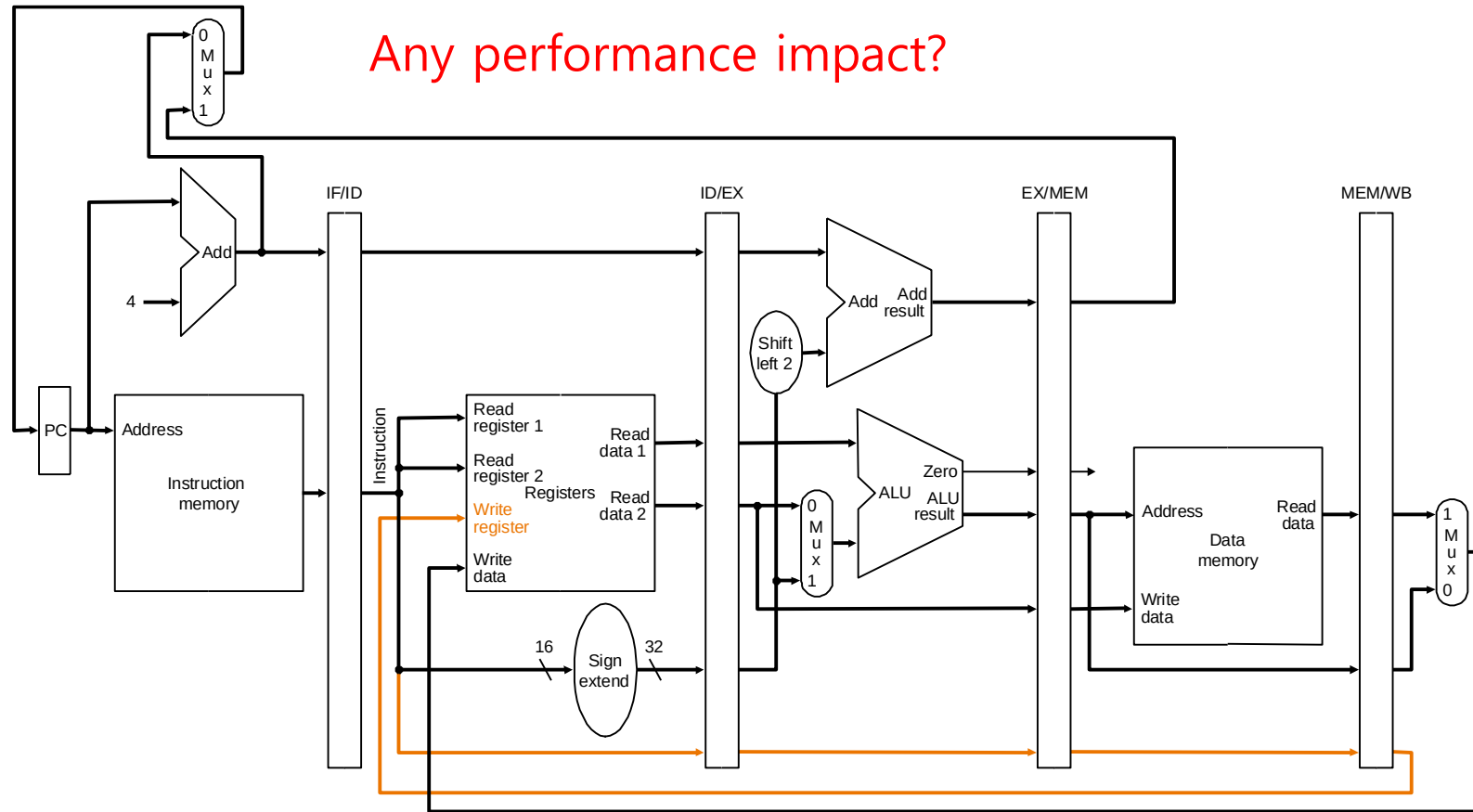
Enabling Pipelined Processing: Pipeline Registers



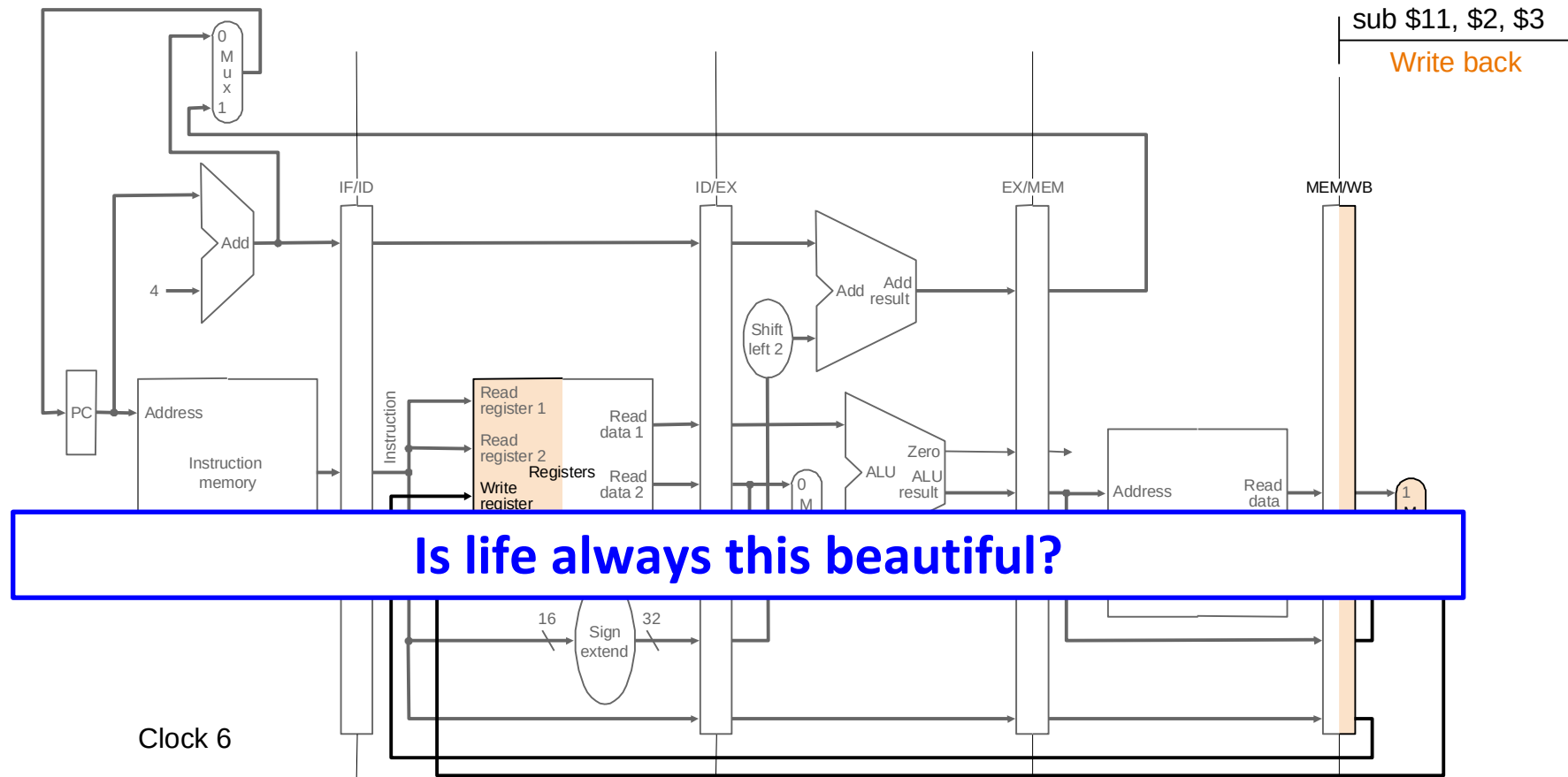
Pipelined Operation Example

All instruction classes must follow the same path and timing through the pipeline stages.

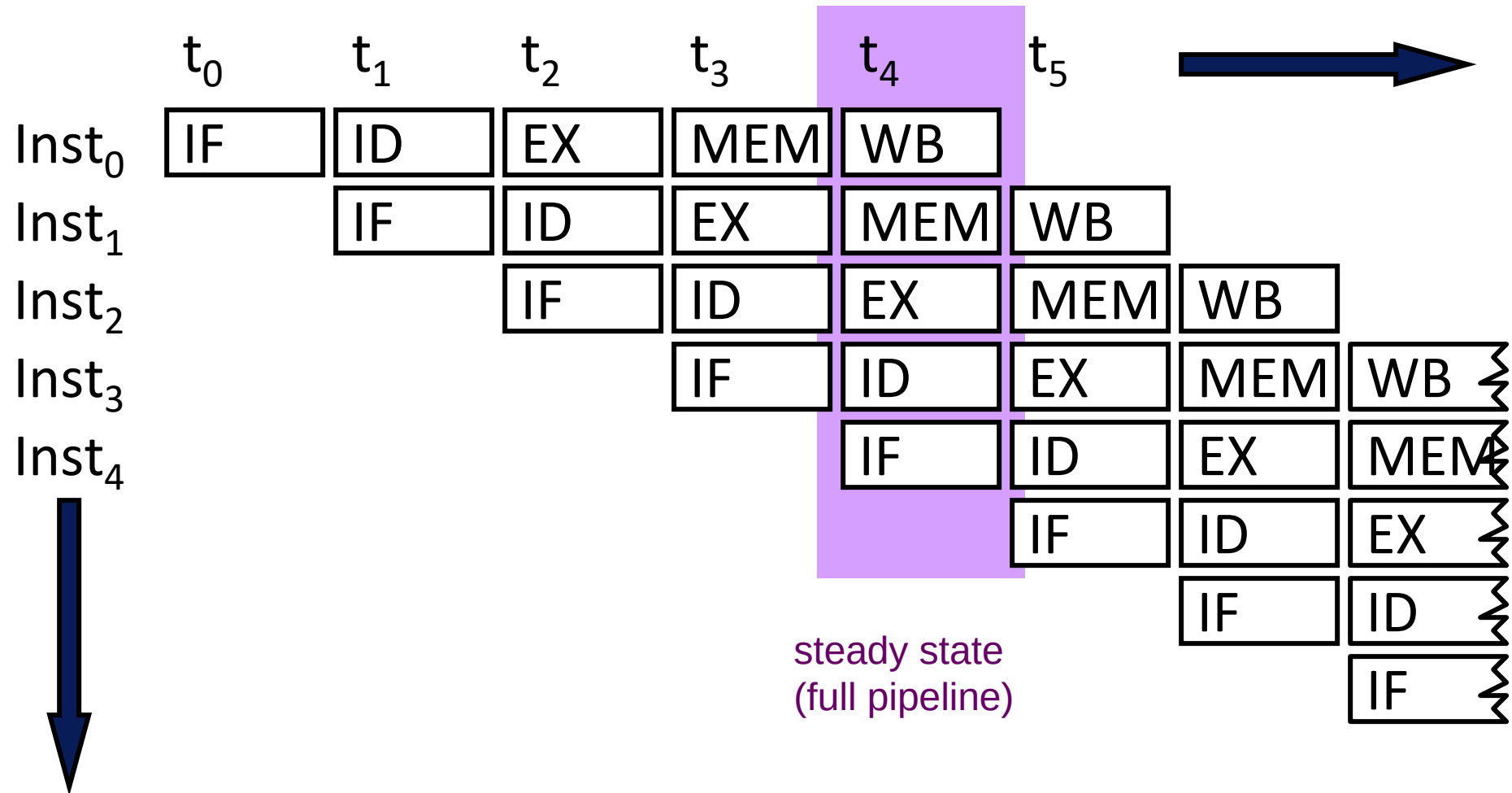
Any performance impact?



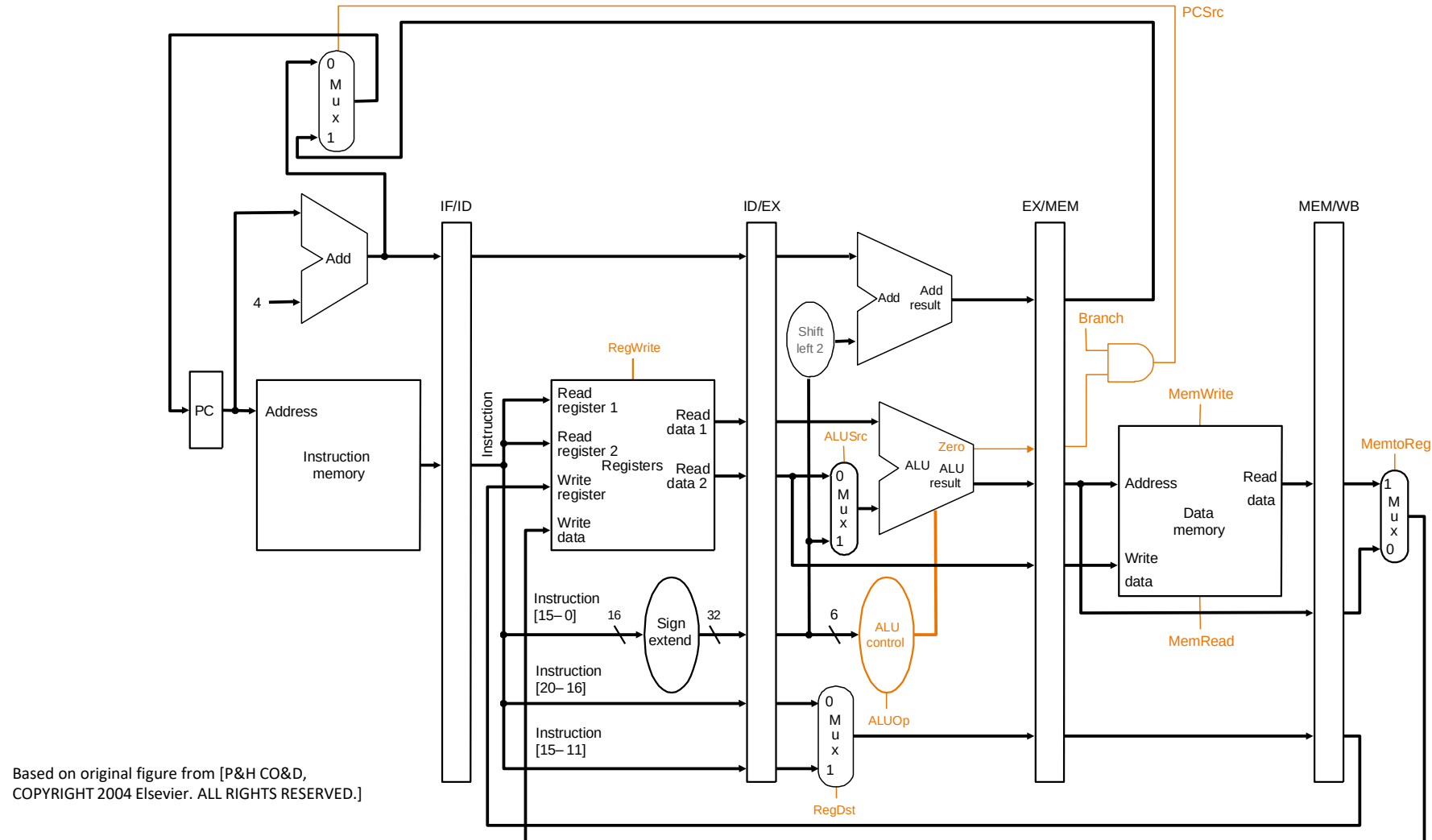
Pipelined Operation Example



Illustrating Pipeline Operation: Operation View



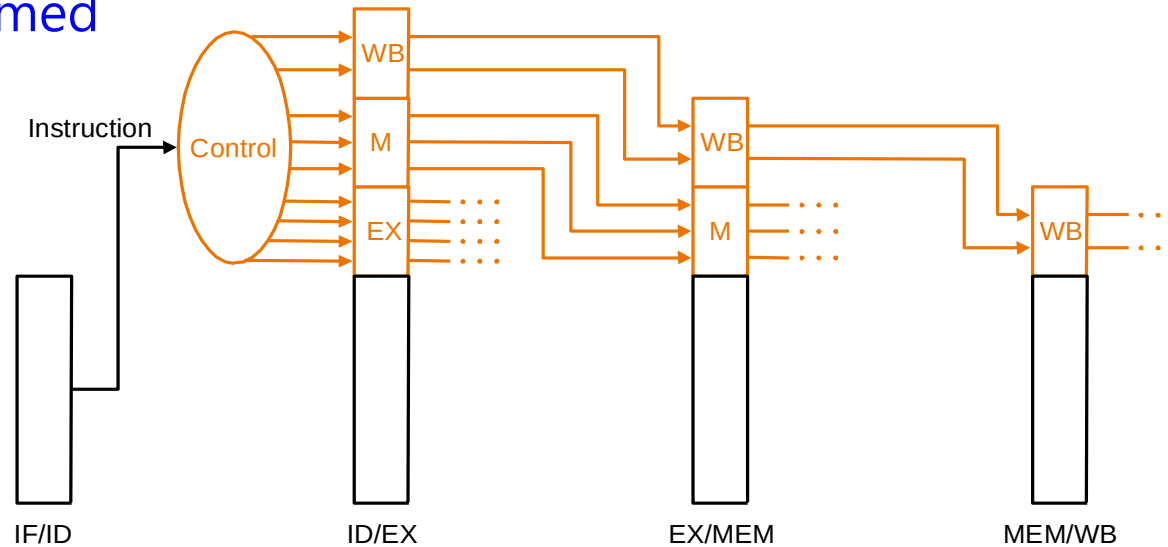
Control Points in a Pipeline



Identical set of control points as the single-cycle datapath!!

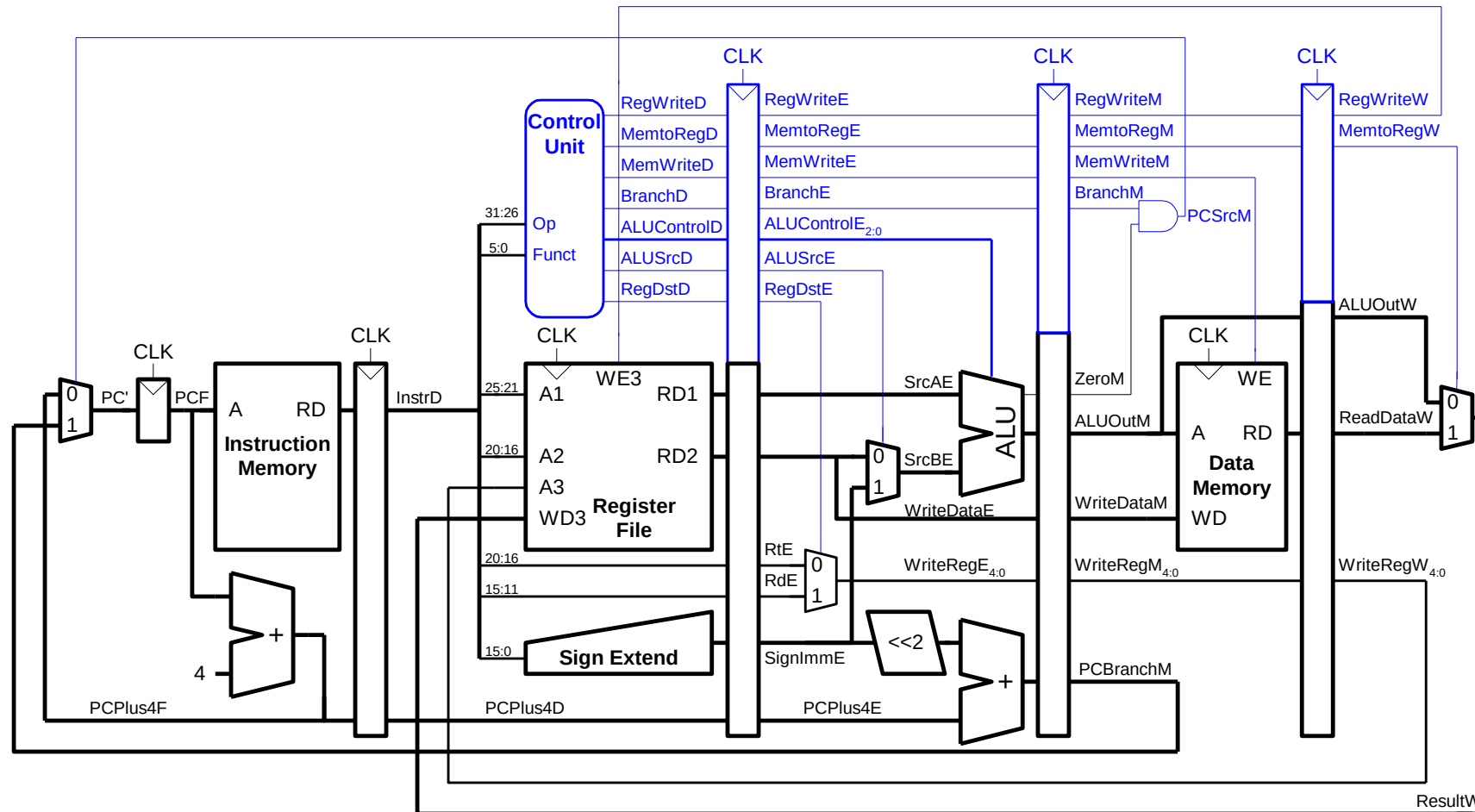
Control Signals in a Pipeline

- For a given instruction
 - same control signals as single-cycle, but
 - control signals required at different cycles, depending on stage
- ⇒ Option 1: decode once using the same logic as single-cycle and buffer signals until consumed
- 
- The diagram shows a single stage of a multi-cycle processor. It consists of a rectangular block with a feedback loop on the left side and an output line on the right side. The feedback loop is represented by an orange arrow that starts from the output, goes up and around the left side of the block, and then points back into the input of the block. The output line is a horizontal orange arrow pointing to the right, away from the block.



⇒ Option 2: carry relevant “instruction word/field” down the pipeline and decode locally within each or in a previous stage

Example: Pipelined Control



- Same control unit as single-cycle processor
Control delayed to proper pipeline stage

Remember: An Ideal Pipeline

- Goal: **Increase throughput with little increase in cost** (hardware cost, in case of instruction processing)
- Repetition of **identical operations**
 - The same operation is repeated on a large number of different inputs (e.g., all laundry loads go through the same steps)
- Repetition of **independent operations**
 - No dependencies between repeated operations
- **Uniformly partitionable suboperations**
 - Processing can be evenly divided into uniform-latency suboperations (that do not share resources)
- Fitting examples: automobile assembly line, doing laundry
 - **What about the instruction processing "cycle"?**

Instruction Pipeline: Not An Ideal Pipeline

■ Identical operations ... NOT!

⇒ different instructions → not all need the same stages

Forcing different instructions to go through the same pipe stages

→ external fragmentation (some pipe stages idle for some instructions)

■ Uniform suboperations ... NOT!

⇒ different pipeline stages → not the same latency

Need to force each stage to be controlled by the same clock

→ internal fragmentation (some pipe stages are too fast but all take the same clock cycle time)

■ Independent operations ... NOT!

⇒ instructions are not independent of each other

Need to detect and resolve inter-instruction dependencies to ensure the pipeline provides correct results

→ pipeline stalls (pipeline is not always moving)

Issues in Pipeline Design

- Balancing work in pipeline stages
 - How many stages and what is done in each stage
- Keeping the pipeline correct, moving, and full in the presence of events that disrupt pipeline flow
 - Handling dependences
 - Data
 - Control
 - Handling resource contention
 - Handling long-latency (multi-cycle) operations
- Handling exceptions, interrupts
- Advanced: Improving pipeline throughput
 - Minimizing *stalls*